Harness Engineering for Autonomous AI Agents: A Comprehensive Survey of Academic Research and Industrial Practice (2026)

Liam Ning*, Claude Code and GLM-5.1

May 4, 2026

Abstract

As Large Language Models (LLMs) evolve from conversational interfaces to autonomous entities capable of long-horizon tasks (Wang et al., 2024b; Xi et al., 2024), the critical performance bottleneck has moved from the model's weights to its **Harness**. Harness Engineering is the systematic discipline of designing the infrastructure, state management systems, and verification protocols that enable probabilistic models to function as reliable software agents. This survey categorizes the rapid advancements of the past two years, bridging the gap between academic research on test-time scaling (Snell et al., 2024; OpenAI, 2024) and engineering technical reports on production-grade systems like the Model Context Protocol (MCP) (Anthropic, 2024b). We provide a framework for the "Agentic Industrial Brain," focusing on deterministic control loops for non-deterministic intelligence. Our contributions include: (1) a formal taxonomy of harness architectures spanning orchestration, tool interfaces, and context management; (2) a comprehensive review of test-time scaling, long-horizon planning, and multi-agent coordination from the academic literature; (3) an analysis of production systems including the Plan-Execute-Verify cycle, sandboxed runtimes, and agentic observability; and (4) a forward-looking analysis of challenges including the "Agentic OS," evaluation standardization, and harness-layer security.

Contents

*csningli.com

1 Introduction

The transition from "Prompting" to "Harnessing" represents the maturation of the AI field (Masterman et al., 2024). In 2024, the community realized that a 7B parameter model within a sophisticated harness often outperforms a 400B parameter model in a naive wrapper. This counterintuitive finding—that infrastructure matters more than raw model capacity for agentic tasks—has catalyzed an entirely new engineering discipline.

1.1 Background and Motivation

The evolution of LLM-based systems has proceeded through three distinct phases, each defined by where the primary engineering effort concentrates.

Phase 1: Model-Centric Engineering (2020–2022). The initial wave of applied LLM work focused on model selection and fine-tuning. Systems like GPT-3 (Brown et al., 2020) and Codex (Chen et al., 2021) demonstrated that scale alone could unlock emergent capabilities, but deployment remained simple: a single API call wrapped in a thin prompt template. The engineering challenge was primarily about curating training data and managing inference costs.

Phase 2: Prompt Engineering (2022–2024). The discovery of Chain-of-Thought prompting (Wei et al., 2022), few-shot learning, and instruction following (Ouyang et al., 2022) shifted focus to the prompt as the primary interface. Techniques like Least-to-Most prompting (Zhou et al., 2023), Self-Consistency (Wang et al., 2023b), and Decomposed Prompting (Khot et al., 2023) demonstrated that reasoning capability could be elicited through careful prompt design. However, these approaches remained fundamentally *single-turn*: the model received a prompt, produced output, and the interaction ended.

Phase 3: Harness Engineering (2024–present). The emergence of autonomous agents—systems that plan, execute multi-step workflows, use tools, and recover from errors—demanded an entirely new engineering layer. The harness mediates between the non-deterministic intelligence of the LLM and the deterministic requirements of real-world systems. Pioneering systems like WebGPT (Nakano et al., 2022), ReAct (Yao et al., 2023b), and Voyager (Wang et al., 2024a) demonstrated that the harness—not the model weights—is the primary determinant of agentic performance.

The motivation for this survey is straightforward: while hundreds of papers have been published on individual components of agent systems (tool use, memory, planning, verification), no comprehensive framework exists that treats the harness as a first-class engineering artifact. This gap is significant because production deployments consistently report that 70–90% of their engineering effort is spent on harness infrastructure rather than model interaction (Masterman et al., 2024; Wang et al., 2024d).

1.2 The Definition of an Agent Harness

We define the **Agent Harness** as the sum of all deterministic code, configuration files, and runtime environments that mediate between the User, the LLM, and the External World. Formally, the harness $\mathcal{H}$ is a tuple:

$$\mathcal{H} = \langle \mathcal{P}, \mathcal{T}, \mathcal{S}, \mathcal{C}, \mathcal{G} \rangle \tag{1}$$

where $\mathcal{P}$ is the *prompt template system* (including system prompts, few-shot examples, and instruction hierarchies), $\mathcal{T}$ is the *tool interface layer* (function schemas, API bindings, and the Model Context Protocol (Anthropic, 2024b)), $\mathcal{S}$ is the *state store* (conversation history, working memory, and long-term knowledge bases), $\mathcal{C}$ is the *control loop* (the orchestration logic that determines when and how the LLM is invoked), and $\mathcal{G}$ is the *guardrail system* (output validation, safety filters, and human-in-the-loop checkpoints).

Figure 1 illustrates the harness architecture as a layered system. The User interacts with the harness through a structured interface; the harness manages all communication with the LLM, maintains state across interactions, and orchestrates tool calls to external systems. Critically, the LLM never directly accesses external resources—all interaction is mediated through the harness's tool interface layer.

This definition distinguishes the harness from both the model (which is a statistical artifact) and the application (which is the end-user experience). The harness is the engineering layer that makes the model's probabilistic outputs safe, reliable, and useful for real-world tasks.

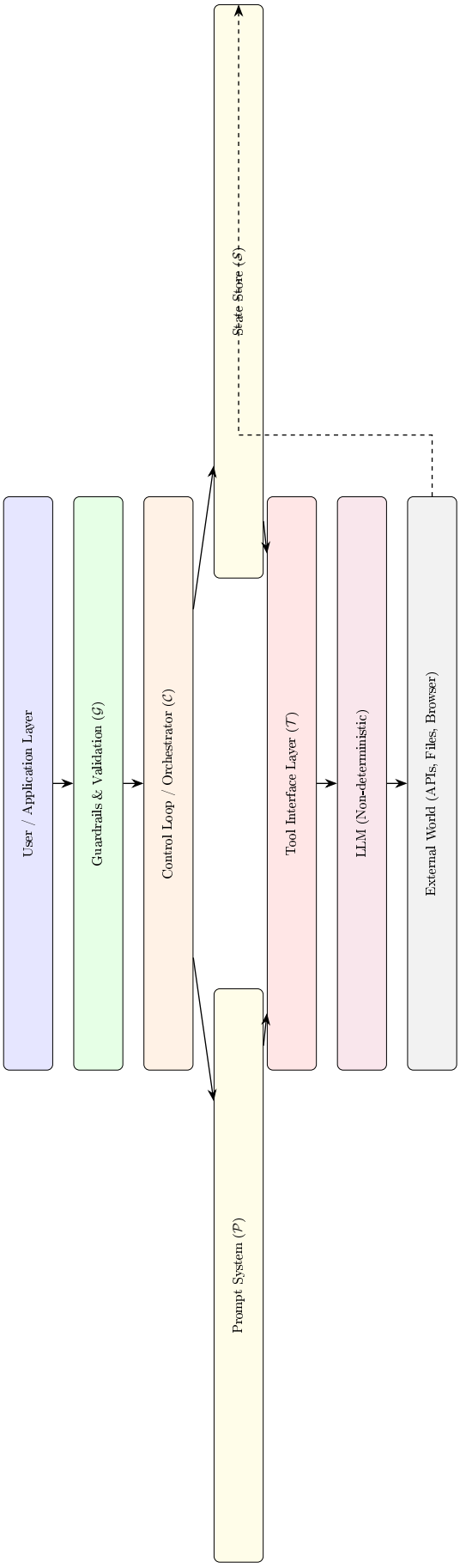

Figure 1: The Agent Harness Architecture. The harness mediates all interaction between the user and the LLM, managing state, orchestrating tool calls, and enforcing safety guardrails. The dashed arrow represents feedback from the external world being stored back into the state store.

1.3 Methodological Scope

This survey examines academic papers from ICLR, NeurIPS, ICML, ACL, EMNLP, and ArXiv (2024–2026) alongside engineering documentation from industry leaders (OpenAI, Anthropic, Cursor, and the OpenHands project (Wang et al., 2024d)).

Our inclusion criteria are as follows:

1. **Agentic architecture**: The paper must describe a system where an LLM autonomously makes decisions across multiple steps, not merely generates a single response.
2. **Infrastructure focus**: The paper must address at least one component of the harness tuple $\mathcal{H} = \langle \mathcal{P}, \mathcal{T}, \mathcal{S}, \mathcal{C}, \mathcal{G} \rangle$.
3. **Evaluation**: The paper must include empirical evaluation, either through established benchmarks or novel task suites.
4. **Reproducibility**: Preference is given to open-source systems where the harness code is publicly available.

We deliberately exclude pure model training papers (e.g., new pre-training techniques), prompt engineering benchmarks that do not involve multi-step orchestration, and proprietary systems without published technical details.

1.4 Contributions and Paper Organization

The key contributions of this survey are:

1. A formal **taxonomy of harness architectures** (Section 3), categorizing orchestration paradigms, tool interfaces, and context management strategies.
2. A comprehensive **academic review** (Section 4) of test-time scaling, long-horizon planning, and multi-agent coordination from 2024–2026.
3. An analysis of **production engineering practices** (Section 5), including the PEV cycle, sandboxed runtimes, and agentic observability.
4. A forward-looking discussion of the **Solopreneur's Industrial Brain** (Section 6), connecting harness engineering to personal productivity frameworks.
5. An identification of **open challenges** (Section 7) including the Agentic OS, evaluation standardization, and harness-layer security.

The remainder of this paper is organized as follows. Section 2 surveys related work and positions our contributions. Section 3 presents our taxonomy of harness architectures. Section 4 reviews the academic literature. Section 5 analyzes production engineering practices. Section 6 explores the Solopreneur's Industrial Brain paradigm. Section 7 discusses challenges and future directions. Section 8 concludes.

2 Related Work

This survey builds on and extends several lines of prior research. We position our work relative to existing surveys, formal frameworks, and system architectures.

2.1 Surveys on LLM-Based Agents

Several surveys have examined LLM-based agents from different perspectives. Wang et al. (2024b) provided a comprehensive survey covering agent architecture profiles, including the modules of brain, perception, and action. Their work focuses on the single-agent paradigm and does not systematically address the harness as an engineering artifact. Xi et al. (2024) surveyed the rise and potential of LLM-based agents, covering architectural foundations, application scenarios, and evaluation approaches. Their taxonomy is organized around agent capabilities rather than harness components.

Masterman et al. (2024) analyzed emerging AI agent architectures specifically for reasoning, planning, and tool use. Their work provides a practitioner-oriented landscape analysis but does not develop a formal taxonomy or address the engineering practices that we examine in Section 5. Our survey differs from these prior works in three key ways: (1) we formally define the harness as a first-class engineering artifact; (2) we bridge academic research with production engineering practices; and (3) we introduce the Solopreneur's Industrial Brain as a novel application paradigm.

2.2 Formal Frameworks for Agentic Systems

The formalization of agent architectures has a rich history spanning classical AI and modern LLM-based systems. The Belief-Desire-Intention (BDI) model from classical agent theory provides a philosophical foundation for understanding agent behavior, where beliefs represent the agent's knowledge, desires represent its goals, and intentions represent its committed plans. Our harness tuple $\mathcal{H} = \langle \mathcal{P}, \mathcal{T}, \mathcal{S}, \mathcal{C}, \mathcal{G} \rangle$ can be viewed as a modern operationalization of the BDI framework: the state store $\mathcal{S}$ corresponds to beliefs, the prompt system $\mathcal{P}$ encodes desires through instructions, and the control loop $\mathcal{C}$ manages intentions through plan execution.

The ReAct framework (Yao et al., 2023b) introduced a formal operational model for LLM-based agents that alternates between reasoning and acting. Our work extends this model by identifying the full set of harness components that must be engineered around the ReAct loop to make it production-ready, including tool interfaces, state management, and guardrails.

2.3 Tool Use and Function Calling

The study of tool use in LLMs has progressed rapidly. Schick et al. (2023) demonstrated that language models can teach themselves to use tools through self-supervised learning. Patil et al. (2023) showed that models can be trained to interact with thousands of real-world APIs. Qin et al. (2024) created ToolLLM, a system for evaluating and training LLMs to master over 16,000 real-world APIs.

Our survey contributes to this line of work by analyzing tool interfaces from the harness engineering perspective—focusing not on the model's ability to use tools but on the infrastructure required to make tool use reliable, secure, and composable. The Model Context Protocol (Anthropic, 2024b) represents a significant advance in this direction, and our analysis in Section 3.2 provides the first systematic comparison of MCP with alternative tool interface standards.

2.4 Software Engineering Agents

The application of LLM-based agents to software engineering has produced a rich body of work. Chen et al. (2021) demonstrated that LLMs fine-tuned on code can achieve significant performance on code generation benchmarks. Austin et al. (2021) introduced HumanEval, a benchmark for evaluating functional correctness of generated code that became the standard evaluation metric for code generation models.

The evolution from code generation to software engineering agents involved several key steps. Code generation models produce code from natural language specifications but do not interact with the execution environment. Software engineering agents, by contrast, navigate codebases, understand project structure, execute tests, and iteratively debug their solutions.

Cognition Labs (2024) announced Devin, described as the "first AI software engineer," which demonstrated end-to-end software engineering capabilities including planning, coding, debugging, and deployment. While the technical details remain proprietary, Devin's public demonstrations highlighted the

importance of the harness: the system includes a browser, code editor, and terminal, all managed by a sophisticated orchestration layer.

Zheng et al. (2024) introduced OpenCodeInterpreter, an open-source system that integrates code generation with execution and iterative refinement in a sandboxed environment. Their work demonstrates the importance of the sandboxing layer (Section 5.2) and the feedback loop between code execution and code generation.

2.5 Knowledge-Augmented Generation

The integration of external knowledge into LLM generation has evolved from simple RAG (Lewis et al., 2020) to sophisticated knowledge management systems. Borgeaud et al. (2022) demonstrated RETRO, which retrieves from a database of trillions of tokens at generation time. Asai et al. (2024) introduced Self-RAG, where the model learns to critique its own retrieval decisions.

From a harness engineering perspective, knowledge-augmented generation is fundamentally a state management problem: the harness must decide what information to retrieve, how to present it to the model, and how to update the knowledge store based on the agent's interactions. The RAG architecture maps directly to the state store component $\mathcal{S}$, with the retrieval system serving as the bridge between the external knowledge base and the LLM's context window.

2.6 Alignment and Safety

The alignment of LLMs with human values and intentions has been a major research focus since 2022. Ouyang et al. (2022) demonstrated that RLHF can significantly improve instruction following while reducing harmful outputs. Bai et al. (2022) introduced Constitutional AI, which trains models to critique their own outputs against a set of principles.

Carlini et al. (2021) demonstrated that LLMs can memorize and reproduce training data, raising privacy concerns that are particularly acute for agent systems that process sensitive user data. Zou et al. (2023) showed that universal adversarial attacks can be crafted to manipulate aligned models, highlighting the need for defense-in-depth approaches.

These alignment challenges are amplified in agent systems. Unlike chatbots that produce a single response, agents execute multi-step workflows where each step can amplify alignment failures. A single misaligned action in a long trajectory can cause disproportionate harm. The harness guardrail system $\mathcal{G}$ must therefore provide continuous alignment verification throughout the agent's execution, not just at the initial prompt level.

3 Taxonomy of Harness Architectures

The architecture of an agent harness determines its expressiveness, reliability, and scalability. In this section, we categorize harness architectures along three axes: orchestration paradigm (how control flows through the system), tool interface design (how the agent interacts with external resources), and context management strategy (how information is maintained across the agent's lifetime).

3.1 Orchestration and Control Loops

The orchestration layer $\mathcal{C}$ defines how the harness invokes the LLM, processes its outputs, and determines the next action. We identify three fundamental paradigms that have evolved over the past two years.

3.1.1 Linear ReAct Loops

The simplest orchestration pattern is the *Reason-Act-Observe* cycle introduced by Yao et al. (2023b). In this paradigm, the agent proceeds through a fixed loop:

1. **Reason**: The LLM generates a textual analysis of the current state and determines the next action.
2. **Act**: The harness extracts a tool call from the LLM's output and executes it.
3. **Observe**: The tool's output is appended to the conversation history and the cycle repeats.

The ReAct loop's primary advantage is its simplicity and debuggability. Each step produces a human-readable trace, and errors can be attributed to specific reasoning steps. However, linear loops struggle with tasks that require backtracking, parallel exploration, or conditional branching.

Variations on the linear loop include the *Scratchpad* approach (Nye et al., 2021), where intermediate computation is explicitly recorded, and the *Plan-and-Solve* framework (Wang et al., 2023a), which decomposes the linear loop into an explicit planning phase followed by sequential execution.

3.1.2 DAG-Based Orchestration

Directed Acyclic Graph (DAG) orchestration generalizes the linear loop by allowing multiple reasoning paths to be explored simultaneously. The key insight is that many agentic tasks can be decomposed into sub-tasks with dependencies, enabling parallel execution and early termination of unpromising branches.

Besta et al. (2024) introduced Graph of Thoughts (GoT), which represents the reasoning process as a directed graph where nodes are LLM-generated thoughts and edges represent transformations or aggregations. This allows the system to:

- **Branch**: Generate multiple candidate solutions in parallel.
- **Merge**: Combine the best elements from multiple branches.
- **Refine**: Iteratively improve specific nodes without regenerating the entire chain.

The Tree of Thoughts (Yao et al., 2023a) is a special case of DAG orchestration where the graph is constrained to a tree structure, enabling efficient search algorithms like breadth-first search (BFS) and depth-first search (DFS) over the reasoning space.

Kim et al. (2024) introduced the LLM Compiler, which applies classical compiler optimization techniques to LLM function calling. The system analyzes the dependency graph of tool calls, identifies independent operations, and executes them in parallel while maintaining correct ordering for dependent calls. This approach achieves up to 3.7× latency reduction compared to sequential function calling.

3.1.3 Hierarchical Multi-Agent Orchestration

The most complex orchestration paradigm distributes control across multiple specialized agents, each with its own prompt system, tools, and state. Hierarchical orchestration introduces a manager-worker topology where a *planner agent* decomposes tasks and delegates sub-tasks to *specialist agents*.

Wu et al. (2023) introduced AutoGen, a framework for multi-agent conversations where agents communicate through structured message passing. The framework supports customizable agent roles, flexible conversation topologies (one-to-one, group chat, hierarchical), and human-in-the-loop participation.

Hong et al. (2024) proposed MetaGPT, which encodes Standardized Operating Procedures (SOPs) into the multi-agent framework. Each agent is assigned a specific role (Product Manager, Architect,

Engineer, QA) with defined inputs, outputs, and verification criteria. The SOP structure reduces the hallucination and drift that plague unconstrained multi-agent systems.

Qian et al. (2024) demonstrated the ChatDev framework for software development, where agents role-play as different members of a software company. The hierarchical structure—with distinct phases for designing, coding, testing, and reviewing—provides natural checkpoints for quality assurance.

Table 1 summarizes the key differences between these orchestration paradigms.

Table 1: Comparison of orchestration paradigms for agent harnesses.

Paradigm	Example	Expressiveness	Latency	Error Recovery
Linear Loop	ReAct (Yao et al., 2023b)	Low	Low	Single-step retry
Linear Loop	Plan-and-Solve (Wang et al., 2023a)	Medium	Medium	Re-planning
DAG-Based	Tree of Thoughts (Yao et al., 2023a)	High	High	Branch pruning
DAG-Based	Graph of Thoughts (Besta et al., 2024)	Very High	High	Merge & refine
DAG-Based	LLM Compiler (Kim et al., 2024)	High	Low	Parallel retry
Hierarchical	AutoGen (Wu et al., 2023)	Very High	Medium	Agent substitution
Hierarchical	MetaGPT (Hong et al., 2024)	Very High	High	SOP checkpoints
Hierarchical	ChatDev (Qian et al., 2024)	High	High	Review phases

3.2 The Model Context Protocol (MCP)

The tool interface layer $\mathcal{T}$ connects the agent to external resources. Until 2024, tool interfaces were ad-hoc: each framework (LangChain, AutoGen, LlamaIndex) defined its own tool schema, making it impossible to share tools across frameworks or reuse tools in different agent systems.

3.2.1 Architecture and Design Philosophy

The Model Context Protocol (MCP) (Anthropic, 2024b), introduced by Anthropic in late 2024, is an open standard for connecting AI agents to external tools and data sources. MCP follows a client-server architecture:

- **MCP Hosts**: The LLM application (e.g., Claude Desktop, Cursor IDE) that initiates connections.
- **MCP Clients**: Protocol clients within the host application that maintain connections to servers.
- **MCP Servers**: Lightweight programs that expose specific capabilities (tools, resources, prompts) through the standardized protocol.

The design philosophy of MCP draws from the Language Server Protocol (LSP) used in IDEs: decouple the tool implementation from the agent framework, enabling a single tool server to serve any MCP-compatible agent. This decoupling has profound implications for the harness ecosystem:

1. **Composability**: Tools can be mixed and matched without code changes.
2. **Security**: Tool servers run in isolated processes with explicit permission grants.
3. **Discoverability**: Tools self-describe their schemas, enabling dynamic tool selection.

3.2.2 Tool Registration, Discovery, and Invocation

MCP defines three primitive types that servers can expose:

1. **Tools**: Functions that the agent can invoke (analogous to function calling). Each tool has a JSON Schema describing its parameters and return type.
2. **Resources**: Structured data that the agent can read (analogous to GET endpoints). Resources represent file-like data such as database records, API responses, or configuration files.
3. **Prompts**: Reusable prompt templates that servers can provide to guide agent behavior for specific tasks.

The discovery process is automatic: when an MCP client connects to a server, it receives a manifest of available tools, resources, and prompts. The harness can then dynamically construct the tool descriptions that are injected into the LLM's context window.

3.2.3 Comparison with Alternative Tool Interfaces

Prior to MCP, the primary tool interface mechanisms were OpenAI's Function Calling API (Anthropic, 2024c) and LangChain's Tool abstraction. Table 2 compares these approaches.

Table 2: Comparison of tool interface standards for agent harnesses.

Feature	**MCP**	**OpenAI FC**	**LangChain**	**ToolLLM** (Qin et al., 2024)
Open Standard	Yes	No	Partial	No
Process Isolation	Yes	No	No	No
Dynamic Discovery	Yes	No	Partial	Yes
Schema Validation	JSON Schema	JSON Schema	Pydantic	JSON Schema
Bidirectional	Yes	No	No	No
Server Ecosystem	Growing	Large	Large	Research only

The key differentiator of MCP is its bidirectional communication model and process isolation. Unlike function calling APIs where tools execute within the agent's process, MCP servers run as separate processes, enabling true sandboxing and permission management. This architectural choice aligns with the harness principle that the LLM should never have direct access to external resources.

3.2.4 MCP Transport and Communication Model

MCP supports two transport mechanisms: standard input/output (stdio) and Server-Sent Events over HTTP (SSE). The stdio transport is used for local tool servers that run on the same machine as the agent, while SSE enables remote tool servers accessible over the network.

The communication model is JSON-RPC 2.0, providing a standardized request-response protocol with support for notifications and error handling. Each MCP message includes:

- A method name (e.g., `tools/list`, `tools/call`, `resources/read`).
- Parameters specific to the method.
- A unique request ID for correlation.

This standardized communication model enables the harness to treat all tools uniformly, regardless of their implementation language or runtime environment. The harness's tool interface layer $\mathcal{T}$ simply translates between the LLM's function call format and MCP's JSON-RPC protocol.

3.2.5 Security Model

MCP's security model is based on the principle of explicit consent:

1. **Server approval**: The user must explicitly approve each MCP server before it can be connected to the agent.
2. **Tool approval**: The user must approve the tools offered by each server before they can be invoked.
3. **Invocation approval**: Optionally, the user can require approval for each individual tool invocation.

This layered approval model provides fine-grained control over the agent's capabilities while minimizing user friction. For production systems, the approval model can be configured based on risk: high-risk tools (e.g., file deletion, network access) require per-invocation approval, while low-risk tools (e.g., file reading, search) are pre-approved.

3.2.6 Ecosystem and Adoption

Since its introduction in late 2024, MCP has seen rapid adoption across the AI tool ecosystem. Major integrations include:

- **Development tools**: File system access, Git operations, database queries.
- **Web services**: Web search, API integration, web scraping.
- **Productivity tools**: Notion, Google Drive, Slack, email integration.

- **Cloud services**: AWS, GCP, Azure resource management.

The growing MCP ecosystem validates the protocol's design philosophy: by decoupling tool implementation from the agent framework, it enables a community-driven approach to tool development where any developer can create an MCP server for their service without waiting for agent framework support.

3.3 Dynamic Context Management

The context window is the most constrained resource in an agent harness. Even as context windows have grown from 4K tokens (GPT-3) to 128K tokens (GPT-4) to 1M+ tokens (Gemini 1.5 Pro), agentic tasks consistently push against these limits because agents must maintain conversation history, tool outputs, retrieved documents, and intermediate reasoning states.

3.3.1 The Context Window as a Constrained Resource

Every invocation of the LLM requires constructing a prompt that fits within the context window. For an agent that has taken n steps, the prompt typically contains:

1. **System prompt**: Instructions and role definition (∼500–2000 tokens).
2. **Tool schemas**: Descriptions of available tools (∼100–500 tokens per tool).
3. **Conversation history**: All prior reasoning and observations (∼n × 200–1000 tokens).
4. **Retrieved context**: Documents or data from RAG (∼500–10,000 tokens).

For a typical software engineering agent, after 20 steps the conversation history alone can exceed 10,000 tokens, leaving diminishing room for new tool outputs. The harness must actively manage this resource to prevent context overflow while preserving the information necessary for coherent action.

Approaches to extending the effective context window include architectural innovations like Longformer's sparse attention (Beltagy et al., 2020) and StreamingLLM's attention sinks (Xiao et al., 2024), but these operate at the model level. Harness-level context management strategies are orthogonal and complementary.

3.3.2 Retrieval-Augmented Generation and Memory

Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) is the most widely deployed context management technique in production agent systems. The core idea is to maintain an external knowledge store and retrieve only the most relevant information for each LLM invocation.

In the harness context, RAG operates as part of the state store $\mathcal{S}$. The harness:

1. Converts the agent's current task into a retrieval query.
2. Searches the knowledge store using vector similarity (e.g., cosine similarity over embeddings).
3. Injects the top-k retrieved documents into the LLM's prompt.

Asai et al. (2024) introduced Self-RAG, which teaches the model to critique its own retrieval decisions. The model learns to determine when retrieval is necessary, whether the retrieved passages are relevant, and whether the generated response is supported by the evidence. This self-reflective approach reduces both hallucination and unnecessary retrieval overhead.

Borgeaud et al. (2022) demonstrated RETRO, which augments autoregressive language models with retrieval from a database of trillions of tokens. While RETRO operates at the model level, its architecture—where retrieved neighbors condition the generation process—has influenced harness designs that interleave retrieval with generation.

3.3.3 Summarization and Context Folding

When the conversation history exceeds the context window, the harness must "fold" older context into a compressed representation. The most common approach is *recursive summarization*: use the LLM itself to summarize the conversation history, replacing the full history with the summary.

Context folding introduces a fundamental tradeoff between information preservation and compression ratio. Aggressive summarization may discard critical details (e.g., specific file paths, error messages, or intermediate results), while conservative summarization fails to achieve sufficient compression. Production systems typically employ a tiered approach:

1. **Working memory**: The last k interactions are kept in full.
2. **Compressed history**: Older interactions are summarized into bullet points.
3. **Long-term storage**: Important facts and decisions are extracted into a structured knowledge base.

This three-tier architecture mirrors the human cognitive distinction between working memory, episodic memory, and semantic memory, and has been adopted by systems like MemGPT (Packer et al., 2024).

3.3.4 Long-Term Memory Architectures

Packer et al. (2024) introduced MemGPT, which treats the context management problem as a virtual memory system analogous to operating systems. In MemGPT, the LLM acts as the "CPU," the context window is "RAM," and external storage (databases, file systems) is "disk." The model learns to manage its own memory through explicit "memory management" function calls—reading from and writing to external storage as needed.

Park et al. (2023) demonstrated Generative Agents, which maintain a comprehensive memory stream of all observations and reflections. The system uses three mechanisms:

- **Memory retrieval**: A scoring function that combines recency, importance, and relevance to select the most pertinent memories.
- **Reflection**: Periodic synthesis of memories into higher-level insights.
- **Planning**: Generation and revision of daily plans based on memories and reflections.

These memory architectures are not mutually exclusive with RAG; production harnesses typically combine retrieval-based access to large knowledge stores with structured memory streams for recent interactions and reflection-based consolidation for long-term learning.

3.4 The Interplay Between Model Capabilities and Harness Design

A recurring theme in harness engineering is that the optimal harness design depends on the capabilities of the underlying model. As models improve, some harness components may become unnecessary; as new model capabilities emerge, new harness components may be required. We analyze this interplay along several dimensions.

3.4.1 Instruction Following and Prompt Complexity

Early LLMs (GPT-3 era) required elaborate prompts with numerous examples and explicit formatting instructions. The prompt system $\mathcal{P}$ was consequently complex, often containing thousands of tokens of examples and formatting rules. Modern instruction-tuned models (Ouyang et al., 2022) require far less prompting: a clear natural language description of the task often suffices.

This evolution has simplified the prompt system but shifted complexity to other harness components. As models become better at understanding natural language instructions, the challenge moves from "how to make the model understand" to "how to ensure the model does what you want"—a guardrail problem rather than a prompt engineering problem.

3.4.2 Reasoning Capability and Orchestration Complexity

The advent of test-time scaling (Snell et al., 2024; OpenAI, 2024) has fundamentally changed the relationship between model capability and orchestration design. When models can "think" before acting, the harness must account for hidden reasoning that consumes tokens and latency without producing visible actions.

This creates a design tension: should the harness implement its own reasoning infrastructure (MCTS, tree search) or delegate reasoning to the model? The answer depends on the task:

- For tasks where the model's built-in reasoning is sufficient (e.g., straightforward question answering), delegating to the model is more efficient.
- For tasks where reasoning must be guided by domain-specific constraints (e.g., software engineering with project conventions), harness-level reasoning provides more control.

- For tasks where reasoning quality must be verified before action (e.g., medical diagnosis), a hybrid approach that combines model reasoning with harness-level verification is optimal.

3.4.3 Context Window Size and Memory Architecture

The rapid expansion of context windows—from 4K tokens to 128K tokens to 1M+ tokens in three years—has profound implications for memory architecture design. In the 4K token era, sophisticated memory management (RAG, summarization, MemGPT) was essential for any non-trivial agent task. In the 1M+ token era, many tasks that previously required complex memory management can now be handled within a single context window.

However, context window size is not a complete solution to the memory problem for several reasons:

1. **Cost**: Filling a 1M token context window is expensive—potentially $10–50 per LLM invocation with frontier models.
2. **Latency**: Processing 1M tokens takes seconds to minutes, even with optimized inference.
3. **Quality**: LLMs may not effectively utilize information in very long contexts. Research suggests that information in the middle of long contexts is often "lost" or underweighted.
4. **Growth**: Task complexity grows at least as fast as context windows. A software engineering agent that previously needed to read 100 files now needs to read 1,000 files in a larger codebase.

These considerations suggest that memory management will remain a critical harness component even as context windows continue to grow, but the specific techniques will evolve. Rather than aggressive compression and summarization, future memory architectures may focus on intelligent selection (choosing the most relevant information to include) and structured organization (presenting information in a format the model can effectively utilize).

Having examined the three primary axes of harness architecture—orchestration, tool interfaces, and context management—we now present a unified framework that relates these components to the formal harness tuple $\mathcal{H} = \langle \mathcal{P}, \mathcal{T}, \mathcal{S}, \mathcal{C}, \mathcal{G} \rangle$.

Table 3 maps each harness component to its design choices, the research contributions that inform those choices, and the production systems that implement them.

Table 3: Unified framework mapping harness components to design choices and implementations.

Component	Design Choices	Key Research	Production
$\mathcal{P}$: Prompt System	Instruction hierarchy	Wei et al. (2022)	Claude, GPT
	Few-shot templates	Brown et al. (2020)	LangChain
	Role definition	Hong et al. (2024)	MetaGPT
$\mathcal{T}$: Tool Interface	Function calling	Schick et al. (2023)	OpenAI FC
	MCP protocol	Anthropic (2024b)	Claude Desktop
	Code-based actions	Wang et al. (2024c)	OpenHands
$\mathcal{S}$: State Store	Conversation history	Yao et al. (2023b)	All systems
	RAG knowledge base	Lewis et al. (2020)	LlamaIndex
	Virtual memory	Packer et al. (2024)	MemGPT
	Memory stream	Park et al. (2023)	Generative Agents
$\mathcal{C}$: Control Loop	Linear ReAct	Yao et al. (2023b)	LangChain
	DAG-based	Besta et al. (2024)	GoT
	Hierarchical	Wu et al. (2023)	AutoGen
	Search-based	Yao et al. (2023a)	ToT
$\mathcal{G}$: Guardrails	Output validation	—	Constitutional AI
	Safety filters	Bai et al. (2022)	Claude
	Human-in-loop	—	Cursor

This framework reveals several important patterns. First, the research contributions are largely independent: each paper addresses a single component in isolation. The engineering challenge of *integrating* these components into a coherent harness has received far less academic attention. Second, production systems tend to combine multiple design choices within each component (e.g., using both conversation history and RAG for state management). Third, the guardrail component $\mathcal{G}$ has the least academic coverage, suggesting an important area for future research.

3.5 Design Patterns for Harness Composition

Based on our analysis of existing systems, we identify several recurring design patterns for composing harness components.

Pattern 1: The Thin Wrapper. The simplest pattern wraps a single LLM call in minimal infrastructure: a fixed system prompt ($\mathcal{P}$), a function calling interface ($\mathcal{T}$), the conversation history ($\mathcal{S}$), a linear ReAct loop ($\mathcal{C}$), and no guardrails ($\mathcal{G} = \emptyset$). This pattern is common in early-stage prototypes and simple chatbots. It is easy to implement but offers no error recovery, no memory management, and no safety guarantees.

Pattern 2: The Verified Agent. This pattern adds verification to the thin wrapper: the control loop includes a PEV cycle where each action is verified before execution. The guardrail system $\mathcal{G}$ includes output validation and, optionally, human-in-the-loop checkpoints. This pattern is used by SWE-agent (Yang et al., 2024) and OpenHands (Wang et al., 2024d) for software engineering tasks where verification through test execution is natural.

Pattern 3: The Memory-Augmented Agent. This pattern adds sophisticated state management: the state store $\mathcal{S}$ includes both conversation history and a long-term memory system (RAG, MemGPT, or a structured knowledge base). The prompt system $\mathcal{P}$ dynamically injects relevant memories into each LLM invocation. This pattern is used by Voyager (Wang et al., 2024a) (skill library), Generative Agents (Park et al., 2023) (memory stream), and production systems like Cursor (project-level context).

Pattern 4: The Multi-Agent Orchestra. This pattern distributes the harness across multiple specialized agents, each with its own prompt system and tool set, coordinated through a shared control loop and state store. This pattern is used by MetaGPT (Hong et al., 2024), ChatDev (Qian et al., 2024), and AutoGen (Wu et al., 2023). The key engineering challenge is managing inter-agent communication and resolving conflicts.

Pattern 5: The Reflective Agent. This pattern adds self-evaluation and self-correction to the control loop. After each action, the agent evaluates its own performance and generates feedback for future steps. This pattern is used by Reflexion (Shinn et al., 2023) (verbal reinforcement learning) and Self-Refine (Madaan et al., 2023) (iterative improvement). The guardrail system includes the agent's own self-critique as a safety mechanism.

These patterns are not mutually exclusive: production systems often combine elements from multiple patterns. For example, a software engineering agent might use the Verified Agent pattern for code generation, the Memory-Augmented Agent pattern for maintaining project context, and the Reflective Agent pattern for debugging.

4 Academic Review (2024–2026)

The academic landscape of agentic AI has evolved rapidly, with major contributions from ICLR, NeurIPS, ICML, and ACL. We organize this review around three themes that directly impact harness design: test-time scaling, long-horizon planning, and multi-agent coordination.

4.1 Test-Time Scaling Laws (System 2 Reasoning)

The most transformative academic insight of 2024–2025 is that inference-time compute can substitute for training-time compute. Rather than scaling model parameters, systems can allocate additional computation during inference to improve reasoning quality.

4.1.1 The O1 Paradigm and System 2 Reasoning

OpenAI's O1 model (OpenAI, 2024) demonstrated that models trained to "think" before answering can dramatically improve on complex reasoning tasks. The key insight is *test-time scaling*: the model generates an internal chain of reasoning (hidden from the user) before producing its final answer. Longer reasoning chains—measured in tokens or steps—correlate with higher accuracy on mathematical, coding, and scientific reasoning benchmarks.

From a harness engineering perspective, the O1 paradigm shifts the control loop's design. Traditional harnesses assume the LLM produces an action at each step; O1-style reasoning introduces a "think" phase that is invisible to the harness but consumes tokens and latency budget. The harness must account for this hidden computation when managing timeouts, cost budgets, and context windows.

The "System 2" framing draws from Kahneman's dual-process theory, where System 1 is fast, intuitive reasoning and System 2 is slow, deliberate computation. In the harness context, this suggests a *dual-mode control loop*: a fast mode for routine actions (tool calls, simple lookups) and a deliberation mode for complex reasoning (planning, debugging, verification).

4.1.2 Compute-Optimal Inference Scaling

Snell et al. (2024) provided the first rigorous analysis of test-time compute scaling. Their key findings are:

1. **Diminishing returns**: Test-time compute scaling follows a power law, with diminishing accuracy improvements as compute increases.
2. **Problem-dependent optimal compute**: Easy problems require minimal test-time compute; hard problems benefit from up to 256× more inference computation.
3. **Model size tradeoff**: A smaller model with optimal test-time scaling can match or exceed a model that is 14× larger but uses standard inference.

These findings have direct implications for harness design. The harness must implement *adaptive compute allocation*: estimating problem difficulty from the initial prompt and adjusting the inference budget accordingly. This requires a difficulty estimation component within the control loop $\mathcal{C}$, which can be implemented as a lightweight classifier or as a function of the LLM's own confidence assessment.

4.1.3 MCTS and Search-Based Reasoning

Monte Carlo Tree Search (MCTS) has emerged as a powerful orchestration strategy for harnessing test-time compute. The approach treats the reasoning process as a game tree, where each node represents a partial solution and edges represent reasoning steps. The MCTS algorithm balances exploration (trying new reasoning paths) and exploitation (deepening promising paths).

The integration of MCTS into the harness control loop creates a fundamentally different orchestration pattern than the linear ReAct loop. Instead of a single path through the reasoning space, the harness maintains a search tree and allocates compute across multiple branches. Key design decisions include:

- **Value function**: How to evaluate partial solutions. Options include the LLM's own assessment, a separate verifier model, or execution-based evaluation (e.g., running test cases).
- **Expansion strategy**: How to generate new branches. Options include temperature sampling, diverse prompting, or structured decomposition.

- **Pruning criteria**: When to abandon unpromising branches to conserve compute.

The Tree of Thoughts framework (Yao et al., 2023a) provides a general template for MCTS-based reasoning with LLMs. Empirical results show that search-based reasoning consistently outperforms greedy decoding on tasks requiring planning, creative problem solving, and multi-step reasoning.

4.1.4 Chain-of-Thought Verification and Self-Consistency

A complementary line of research focuses on *verifying* reasoning chains rather than scaling their generation. The Self-Consistency approach (Wang et al., 2023b) samples multiple reasoning paths for the same problem and selects the most common answer through majority voting. This simple strategy achieves significant accuracy improvements without any model modifications, making it a popular harness-level technique.

Wei et al. (2022) demonstrated that Chain-of-Thought prompting not only improves accuracy but also makes the reasoning process inspectable. This inspectability is critical for harness engineering: it enables the guardrail system $\mathcal{G}$ to validate the agent's reasoning before executing potentially destructive actions.

The combination of search-based reasoning and verification creates a powerful harness pattern:

1. Generate multiple candidate solutions using MCTS or sampling.
2. Verify each candidate using execution, formal methods, or LLM-based critique.
3. Select the highest-confidence solution.
4. If no candidate passes verification, increase the compute budget and repeat.

This "generate-verify-select" loop is the foundation of many production agent systems, as we discuss in Section 5.

4.2 Long-Horizon Planning and Drift Mitigation

Real-world agentic tasks—writing software, conducting research, managing projects—require agents to maintain coherence across dozens or hundreds of steps. The primary challenge is *trajectory drift*: as the agent executes more steps, it gradually deviates from the original plan, accumulates errors, and loses track of its objectives.

4.2.1 The Accumulated Error Problem

Accumulated error in agentic trajectories manifests in several ways:

1. **Plan drift**: The agent's actions become increasingly disconnected from the original goal.
2. **Context dilution**: Important information from earlier steps is forgotten or overwhelmed by irrelevant details.
3. **Error compounding**: A mistake in step k propagates through steps $k+1, k+2, \ldots$ making recovery increasingly difficult.
4. **Resource exhaustion**: The agent exhausts its context window, compute budget, or tool call quota before completing the task.

The SWE-bench benchmark (Jimenez et al., 2024) quantifies this problem: on tasks requiring more than 10 tool calls, agent success rates drop by 40–60% compared to single-step tasks, even when each individual step is within the model's capability.

4.2.2 Reflexion and Self-Correction Loops

Shinn et al. (2023) introduced Reflexion, a framework that augments agents with verbal reinforcement learning. The key innovation is a *self-evaluation* step after each action: the agent generates a verbal critique of its performance, identifying mistakes and suggesting improvements. These critiques are stored in a "self-reflection" memory and injected into future reasoning steps.

The Reflexion loop extends the ReAct cycle with two additional phases:

1. **Evaluate**: After executing an action, assess whether it achieved the intended result.
2. **Reflect**: If the evaluation is negative, generate a critique explaining what went wrong and how to improve.

From a harness engineering perspective, Reflexion requires the state store $\mathcal{S}$ to maintain not only the conversation history but also a separate reflection memory. The control loop $\mathcal{C}$ must incorporate evaluation checkpoints—either after every step or at strategic points determined by task structure.

4.2.3 Voyager and Skill Library Composition

Wang et al. (2024a) introduced Voyager, an open-ended embodied agent for Minecraft that addresses the accumulated error problem through *skill library composition*. Voyager maintains a growing library of verified skills (code programs) that can be composed to solve increasingly complex tasks.

The skill library operates as a form of procedural memory within the harness:

1. **Skill proposal**: The LLM generates a candidate skill (a JavaScript function) for the current task.
2. **Skill execution**: The skill is executed in the Minecraft environment.
3. **Skill verification**: The environment provides feedback on whether the skill achieved its objective.
4. **Skill storage**: Verified skills are added to the library for future reuse.

The skill library addresses accumulated error by reducing the effective horizon of the agent. Instead of planning 50 steps into the future, the agent decomposes tasks into reusable 5–10 step skills, each of which is independently verified. This decomposition dramatically reduces error compounding because each skill is a verified, reliable building block.

The SWE-agent system (Yang et al., 2024) applies a similar philosophy to software engineering. The agent's "Agent-Computer Interface" (ACI) defines a set of powerful, composable actions (search, edit, test) that serve as the building blocks for complex software engineering workflows.

4.2.4 Self-Refine and Iterative Improvement

Madaan et al. (2023) proposed Self-Refine, a framework where the LLM iteratively improves its own outputs through self-feedback. Unlike Reflexion, which focuses on external evaluation, Self-Refine uses the model's own assessment to drive improvement.

The Self-Refine loop consists of three phases:

1. **Generate**: Produce an initial output.
2. **Feedback**: The model critiques its own output, identifying specific improvements.
3. **Refine**: The model revises the output based on its own feedback.

This loop repeats until the model's feedback indicates satisfaction or a maximum iteration count is reached. The harness implementation is straightforward: the control loop simply re-invokes the LLM with the previous output and the self-generated feedback appended to the context.

Empirical results show that Self-Refine achieves significant improvements on code generation (8–12% absolute improvement on HumanEval (Austin et al., 2021)), text rewriting, and constrained generation tasks. The key limitation is that self-feedback can be unreliable—the model may fail to identify its own errors, or may converge to a local optimum.

4.2.5 Comparative Analysis of Planning Approaches

The planning approaches discussed above can be compared along several dimensions. Table 4 summarizes this comparison.

The choice of planning approach depends on the task characteristics. For open-ended tasks where the agent must explore and learn over time, Voyager's skill library approach is most appropriate. For well-defined tasks with clear success criteria, the PEV cycle with execution-based verification is most reliable. For tasks where the primary risk is reasoning errors rather than planning failures, Self-Refine provides a lightweight self-correction mechanism.

A key open question is whether these approaches can be effectively combined. A hierarchical harness might use Voyager-style skill libraries for long-term learning, the PEV cycle for task-level execution, and Self-Refine for step-level quality improvement. The engineering challenge is managing the complexity of such a multi-layered system without introducing new failure modes.

Table 4: Comparison of long-horizon planning approaches.

Approach	Drift Res.	Error Rec.	Complexity	Cost	Best For
Reflexion (Shinn et al., 2023)	Medium	High	Low	Medium	Sequential tasks
Voyager (Wang et al., 2024a)	High	High	High	High	Open-ended tasks
Self-Refine (Madaan et al., 2023)	Low	Medium	Low	Low	Single-step tasks
PEV (Sec. 5.1)	High	High	Medium	Medium	Production systems

4.2.6 The Role of World Models in Long-Horizon Planning

An emerging line of research explores whether agents can develop internal "world models"—predictive models of how their actions will affect the environment—to improve planning. A world model enables the agent to simulate the consequences of its actions before executing them, reducing the need for trial-and-error exploration.

In the harness context, a world model would operate as part of the control loop $\mathcal{C}$, providing:

1. **Action prediction**: Given the current state and a proposed action, predict the resulting state.
2. **Plan evaluation**: Given a complete plan, predict whether it will achieve the goal.
3. **Counterfactual reasoning**: Given a failed execution, predict what would have happened if a different action had been taken.

World models can be implemented through several mechanisms:

- **LLM-based simulation**: Use the LLM itself to predict the outcome of actions. This leverages the model's extensive world knowledge but may be inaccurate for domain-specific predictions.
- **Learned environment models**: Train a separate model on interaction traces to predict outcomes. This is more accurate but requires training data and may not generalize to novel situations.
- **Formal verification**: Use program analysis or model checking to predict outcomes for code-based actions. This provides exact predictions but is limited to formally tractable domains.

The integration of world models into agent harnesses represents a promising direction for reducing the accumulated error problem, as it enables the agent to detect and avoid potential failures before they occur.

4.3 Multi-Agent Coordination and Consensus

Multi-agent systems distribute cognitive labor across specialized agents, enabling parallelism, role-based expertise, and robustness through redundancy. However, coordination introduces new challenges: communication overhead, consensus formation, and conflict resolution.

4.3.1 Communication Topologies

The topology of inter-agent communication determines both the efficiency and the reliability of the multi-agent system. We identify four primary topologies:

1. **Star topology**: A central orchestrator communicates with all agents. Used by AutoGen (Wu et al., 2023) and most production systems. Simple to implement but creates a bottleneck at the orchestrator.
2. **Ring topology**: Agents communicate sequentially, each building on the previous agent's output. Used by ChatDev (Qian et al., 2024) for sequential software development phases. Low overhead but no parallelism.
3. **Mesh topology**: All agents can communicate with all other agents. Used by debate-based systems where agents critique each other's outputs. High communication cost but enables consensus through argumentation.
4. **Hierarchical topology**: Agents are organized in a tree, with managers delegating to subordinates. Used by MetaGPT (Hong et al., 2024). Balances parallelism with coordination overhead.

4.3.2 Role-Based Specialization

Hong et al. (2024) demonstrated that encoding explicit roles with defined responsibilities significantly improves multi-agent performance. In MetaGPT, agents are assigned standard software engineering roles: the Product Manager writes requirements, the Architect designs the system, the Engineer implements code, and the QA Engineer writes tests. Each role has defined input formats and output formats, creating a structured pipeline that reduces ambiguity and error propagation.

Qian et al. (2024) applies a similar approach with "chat chains"—sequential conversations between role-playing agents that progress through defined phases (designing, coding, testing, reviewing). The chat chain structure ensures that each phase produces verified output before the next phase begins.

Shen et al. (2023) introduced HuggingGPT, which uses a controller agent to parse user requests into sub-tasks, select appropriate specialist models from Hugging Face, and synthesize the results. This "LLM as orchestrator" paradigm treats the LLM as a meta-controller that reasons about which specialized tools (other models) to invoke.

4.3.3 Consensus and Conflict Resolution

When multiple agents produce conflicting outputs, the harness must implement a consensus mechanism. Approaches include:

- **Voting**: Each agent votes on proposed solutions, and the majority wins. Effective when agents have complementary expertise but requires an odd number of voters.
- **Debate**: Agents present arguments for their positions and a judge agent (or human) makes the final decision. More nuanced than voting but slower.
- **Verification-based**: Solutions are independently verified (e.g., by running tests), and only verified solutions are accepted. Eliminates subjectivity but requires a reliable verification oracle.

Table 5 summarizes the characteristics of major multi-agent frameworks.

Table 5: Comparison of multi-agent frameworks for harness coordination.

Framework	Topology	Specialization	Open Source	Domain
AutoGen (Wu et al., 2023)	Star/Mesh	Custom roles	Yes	General
MetaGPT (Hong et al., 2024)	Hierarchical	SE roles	Yes	Software Eng.
ChatDev (Qian et al., 2024)	Ring	SE roles	Yes	Software Eng.
HuggingGPT (Shen et al., 2023)	Star	Model selection	Yes	Multi-modal
CrewAI (CrewAI, 2024)	Star/Mesh	Custom roles	Yes	General

4.4 Code Agents and Software Engineering Benchmarks

Software engineering has emerged as the primary testbed for agentic AI systems, driven by the availability of standardized benchmarks and the structured nature of code as an output medium.

4.4.1 The SWE-bench Paradigm

Jimenez et al. (2024) introduced SWE-bench, a benchmark that evaluates AI agents on their ability to resolve real-world GitHub issues. SWE-bench is constructed from 2,294 Python software engineering tasks, each consisting of a GitHub issue (the task description) and the corresponding pull request (the ground-truth solution). The benchmark evaluates agents on their ability to:

1. Understand the issue description and identify the relevant code.
2. Navigate the codebase to locate the files that need modification.
3. Write correct patches that resolve the issue without introducing regressions.
4. Pass the repository's existing test suite.

SWE-bench has become the de facto standard for evaluating software engineering agents. The key insight from SWE-bench evaluations is that the harness matters more than the model: different harnesses built on the same base model achieve wildly different performance, with the best harnesses (SWE-agent (Yang et al., 2024), OpenHands (Wang et al., 2024d)) significantly outperforming simpler wrappers.

4.4.2 Agent-Computer Interfaces

Yang et al. (2024) introduced the concept of the Agent-Computer Interface (ACI), analogous to the Human-Computer Interface (HCI). The ACI defines the actions available to the agent (e.g., search, open file, edit, run command) and the format of the feedback it receives. The key insight is that the ACI design has a dramatic impact on agent performance: a well-designed ACI can double the success rate compared to a naive interface.

The ACI concept generalizes beyond software engineering. In web browsing, the interface includes clicking, scrolling, and form filling (Zhou et al., 2024). In data analysis, the interface includes query execution, visualization, and statistical testing. In each domain, the ACI design determines the space of actions available to the agent and, consequently, the complexity of the control loop required to navigate that space.

4.4.3 The CodeAct Paradigm

Wang et al. (2024c) proposed CodeAct, which treats code as the universal action space for agents. Instead of defining domain-specific tool interfaces, CodeAct allows the agent to write and execute Python code to accomplish any task. This approach has several advantages:

- **Universality**: Code can express any computable action, eliminating the need for domain-specific tool definitions.
- **Composability**: Complex actions can be composed from simpler ones through function calls, loops, and conditional logic.
- **Debuggability**: Code execution produces standard outputs and error messages that the agent can interpret.
- **Leverage**: The LLM's strong code generation capabilities are directly utilized.

The CodeAct paradigm has been adopted by OpenHands (Wang et al., 2024d) and is increasingly influential in the design of production agent systems.

4.5 Benchmarks and Evaluation Methodology

The evaluation of agent systems requires fundamentally different methodologies than the evaluation of static LLMs. We survey the major benchmarks and their implications for harness engineering.

4.5.1 Agent-Centric Benchmarks

Table 6 summarizes the major benchmarks used to evaluate agent systems.

Table 6: Major benchmarks for evaluating agent systems.

Benchmark	Domain	Tasks	Interactive	Year
SWE-bench (Jimenez et al., 2024)	Software Eng.	2,294	Yes	2024
WebArena (Zhou et al., 2024)	Web Browsing	812	Yes	2024
AgentBench (Liu et al., 2024a)	General	887	Yes	2024
HumanEval (Austin et al., 2021)	Code Gen.	164	No	2021
MATH (Hendrycks et al., 2021)	Mathematics	12,500	No	2021
WebShop (Yao et al., 2022)	E-commerce	Varies	Yes	2022

A critical distinction exists between *static* benchmarks (where the agent produces a single output) and *interactive* benchmarks (where the agent must navigate an environment through multiple steps). Interactive benchmarks are far more demanding of the harness because they test not just the model's reasoning but also the orchestration, state management, and error recovery capabilities of the harness.

4.5.2 Evaluation Metrics for Harness Engineering

Traditional evaluation metrics (accuracy, F1, BLEU) are insufficient for agent systems. We propose a set of harness-specific evaluation metrics:

1. **Task Completion Rate (TCR)**: The fraction of tasks the agent completes successfully. The most basic metric but does not capture efficiency or cost.
2. **Steps-to-Completion (S2C)**: The number of agent steps required to complete a task. Lower is better, indicating more efficient planning and execution.
3. **Token Efficiency (TE)**: The ratio of task completion to total tokens consumed. Captures the cost-effectiveness of the harness.
4. **Error Recovery Rate (ERR)**: The fraction of errors that the agent successfully recovers from without human intervention. Measures the robustness of the control loop.
5. **Plan Adherence (PA)**: The degree to which the agent follows its initial plan, measured as the divergence between planned and actual action sequences. High plan adherence indicates low drift.

These metrics provide a multi-dimensional view of harness performance that goes beyond simple success/failure. Production systems should track all five metrics to identify bottlenecks and guide optimization efforts.

5 Engineering Practice and Production Systems

While academic research explores the frontier of what is possible, production engineering focuses on what is reliable, scalable, and cost-effective. This section analyzes the engineering practices that have emerged from deploying agent systems at scale.

5.1 The PEV (Plan-Execute-Verify) Cycle

The Plan-Execute-Verify (PEV) cycle has emerged as the dominant paradigm for production agent systems. PEV decomposes agentic workflows into three distinct phases, each with its own engineering requirements.

5.1.1 Plan Phase: Task Decomposition

The planning phase decomposes a high-level task into a sequence of executable sub-tasks. Effective planning requires:

1. **Task understanding**: The LLM must comprehend the user's intent, including implicit requirements and constraints.
2. **Decomposition**: The task must be broken into sub-tasks that are individually tractable and collectively sufficient.
3. **Dependency analysis**: Sub-tasks must be ordered to respect dependencies (e.g., "search for the file" must precede "edit the file").
4. **Resource estimation**: The harness must estimate the context window, tool calls, and time required for each sub-task.

Wang et al. (2023a) demonstrated that explicit plan generation improves performance by 10–20% on multi-step reasoning tasks compared to implicit planning through chain-of-thought. The harness implementation typically reserves the first LLM invocation exclusively for planning, producing a structured plan (often in JSON or markdown format) that guides subsequent execution.

5.1.2 Execute Phase: Tool Orchestration

The execution phase carries out the plan through tool invocations. The key engineering challenges are:

- **Tool selection**: Given a sub-task, the harness must select the appropriate tool from its tool library. MCP (Anthropic, 2024b) provides a standardized mechanism for tool discovery and invocation.
- **Error handling**: Tool calls may fail due to network errors, permission issues, or invalid parameters. The harness must implement retry logic, fallback strategies, and graceful degradation.
- **Parallel execution**: Independent sub-tasks can be executed concurrently to reduce latency. The LLM Compiler (Kim et al., 2024) provides a principled approach to parallel function calling.
- **State management**: Each tool call produces output that must be stored and made available for subsequent steps.

The OpenHands framework (Wang et al., 2024d) implements execution through a code-based action space: the agent writes and executes Python code to interact with the environment. This approach leverages the LLM's strong code generation capabilities and provides a universal interface for tool composition. The CodeAct framework (Wang et al., 2024c) extends this idea by treating all agent actions as code execution, unifying tool use, reasoning, and communication within a single programming paradigm.

5.1.3 Verify Phase: Automated Quality Assurance

The verification phase is what distinguishes production agent systems from research prototypes. Verification mechanisms include:

1. **Execution-based verification**: Run the code, check the output, compare against expected results. SWE-bench (Jimenez et al., 2024) uses unit tests as the verification oracle for software engineering agents.

2. **LLM-based verification**: Use a separate LLM (or the same LLM with a different prompt) to critique the agent's output. This is less reliable than execution-based verification but applicable to tasks without objective test suites.

3. **Human-in-the-loop verification**: Present the agent's output to a human for review. Essential for high-stakes tasks where errors are costly or irreversible.

Figure 2 illustrates the PEV cycle as implemented in a typical production harness.

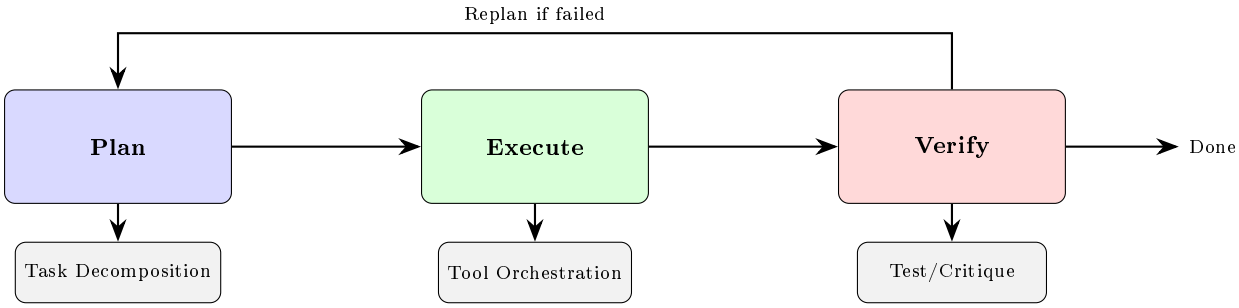

Figure 2: The Plan-Execute-Verify (PEV) cycle in a production agent harness. If verification fails, the cycle loops back to replanning with error information injected into the context.

The PEV cycle's power lies in its iterative nature. When verification fails, the error information is fed back into the planning phase, enabling the agent to revise its approach. This "generate-test-revise" pattern is analogous to test-driven development in software engineering and has been shown to improve task completion rates by 30–50% compared to non-iterative approaches.

5.1.4 Implementation Considerations for the PEV Cycle

Implementing the PEV cycle in a production harness requires careful attention to several engineering details:

Plan Representation. The plan must be represented in a format that is both human-readable (for debugging and oversight) and machine-parseable (for tracking progress). Common representations include:

- **Markdown checklists**: Simple and readable but lack formal structure.
- **JSON task graphs**: Structured and machine-parseable but verbose and less readable.
- **Natural language descriptions**: Flexible but ambiguous; may require re-parsing at each step.

Production systems typically use a hybrid approach: a high-level natural language plan accompanied by a structured JSON representation that tracks completion status, dependencies, and intermediate results.

Error Feedback Integration. When verification fails, the error information must be effectively communicated to the planning phase. The harness must:

1. Capture the full context of the failure (what was attempted, what went wrong, what the error message was).
2. Analyze the failure to identify the root cause (was it a planning error, an execution error, or an environmental issue?).
3. Generate a revised plan that addresses the root cause without repeating the same mistake.

The Reflexion approach (Shinn et al., 2023) provides a template for this error feedback loop: the agent generates a verbal critique of the failure, which is then incorporated into the revised plan.

Iteration Limits and Escalation. The PEV cycle must include safeguards against infinite loops:

1. **Maximum iterations**: A hard limit on the number of PEV cycles per task.
2. **Diminishing returns detection**: If consecutive iterations make no progress (as measured by verification scores), the cycle is terminated.

3. **Human escalation**: When the agent exceeds the iteration limit or makes no progress, the task is escalated to a human with a summary of attempts and failures.

Concurrency and Parallelism. For complex tasks with independent sub-tasks, the PEV cycle can be parallelized:

1. The planner identifies independent sub-tasks.
2. Each sub-task is assigned to a separate PEV cycle running in parallel.
3. The results are merged and verified at the aggregate level.

The LLM Compiler (Kim et al., 2024) provides a principled approach to identifying parallelizable sub-tasks and managing their execution.

5.2 Durable Runtimes and Sandboxing

Agent systems that interact with the real world—executing code, modifying files, making API calls—require robust isolation mechanisms to prevent unintended damage and ensure reproducibility.

5.2.1 E2B Cloud Sandboxes

E2B (E2B, 2024) provides cloud-based sandboxed environments specifically designed for AI agent execution. Each agent session runs in an isolated micro-VM with:

- A complete Linux environment with pre-installed development tools.
- Network isolation with explicit allowlists for external access.
- Filesystem isolation with snapshot and restore capabilities.
- Time-limited execution with automatic cleanup.

The E2B approach integrates with the harness through an SDK that manages the lifecycle of sandbox environments. The harness creates a sandbox for each agent session, routes all tool calls through the sandbox, and destroys the sandbox when the session completes. This architecture ensures that agent actions are fully contained and reproducible.

5.2.2 Docker and Container-Based Isolation

Docker-based isolation is the most common approach for self-hosted agent systems. The SWE-agent (Yang et al., 2024) framework uses Docker containers to provide each agent with an isolated development environment that mirrors real-world software engineering setups.

The container-based approach offers several advantages:

1. **Reproducibility**: The same Docker image produces identical environments across runs.
2. **Resource limits**: CPU, memory, and disk quotas prevent runaway agents from consuming host resources.
3. **Network control**: Docker networking allows fine-grained control over internet access.
4. **Snapshotting**: Container filesystem changes can be committed to images, enabling rollback and analysis.

5.2.3 WASM and Lightweight Sandboxing

WebAssembly (WASM) is emerging as a lightweight alternative to Docker for agent sandboxing. WASM sandboxes boot in milliseconds (vs. seconds for Docker), consume minimal memory, and can run anywhere—including in-browser. This makes WASM particularly suitable for high-throughput agent deployments where cost and latency are critical.

The tradeoff is that WASM sandboxes have more limited system access than Docker containers: they cannot directly access the host filesystem, make arbitrary network calls, or execute native binaries. This limitation is also a security feature—the reduced attack surface makes WASM sandboxes harder to escape.

5.2.4 Security Considerations for Agent Runtimes

Regardless of the sandboxing technology, the harness must implement several security measures:

1. **Principle of least privilege**: Agents should only have access to the resources they need for the current task.
2. **Audit logging**: All agent actions should be logged for forensics and compliance.
3. **Rate limiting**: Tool call frequency and resource consumption should be bounded.
4. **Escape detection**: The harness should monitor for signs that the agent is attempting to break out of its sandbox.

5.3 Agentic Observability and Trajectory Analysis

Traditional software observability (logs, metrics, traces) is insufficient for agent systems because the primary source of failure is not infrastructure but *reasoning errors*. Agentic observability requires new tools and techniques that make the agent's decision-making process transparent.

5.3.1 Beyond Logging: Structured Traces

Agent traces capture the full decision-making process of the agent, including:

- **Reasoning traces**: The LLM's internal chain of thought at each step.
- **Action traces**: The tool calls made, their parameters, and their results.
- **State snapshots**: The contents of the agent's working memory at each step.
- **Cost traces**: Token consumption and latency at each step.

Structured traces enable several critical workflows:

1. **Failure analysis**: When an agent fails a task, the trace reveals exactly where the reasoning went wrong.
2. **Performance optimization**: Token and latency traces identify bottlenecks in the agent's workflow.
3. **Quality assurance**: Traces can be reviewed by humans to ensure the agent's reasoning is sound.

The AgentBench (Liu et al., 2024a) and WebArena (Zhou et al., 2024) benchmarks generate structured traces as part of their evaluation, enabling researchers to analyze not just whether agents succeed but *how* they succeed or fail.

5.3.2 Trajectory Analysis and Replay

Trajectory analysis treats the agent's trace as a data structure that can be queried, compared, and aggregated. Key analyses include:

- **Path comparison**: Comparing the trajectories of successful and failed runs to identify decision points that correlate with success.
- **Pattern mining**: Identifying recurring patterns in agent behavior (e.g., common error recovery strategies).
- **Regression detection**: Comparing trajectories across harness versions to detect unintended behavioral changes.

Trajectory replay allows developers to re-execute specific steps of an agent's run, modify the inputs, and observe the effects. This is invaluable for debugging complex agentic workflows where the interaction between the LLM, tools, and state management produces emergent behavior.

5.3.3 Cost and Latency Profiling

Production agent systems must manage cost and latency as first-class constraints. A typical agentic workflow consumes 10–100× more tokens than a single LLM call, and the latency is dominated by the number of sequential LLM invocations and tool calls.

Cost profiling requires tracking:

- Token consumption per step (input and output tokens separately).
- Tool call latency and success rates.
- Retry frequency and the cost of failed attempts.
- Cache hit rates for prompt caching optimizations.

Latency profiling identifies the critical path through the agent's workflow and highlights opportunities for parallelization. The LLM Compiler's approach (Kim et al., 2024) to parallel function calling can reduce end-to-end latency by 2–4× for tasks with independent sub-tasks.

5.3.4 The Observability Stack

A production-grade observability stack for agent systems typically consists of three layers:

Layer 1: Real-Time Monitoring. Real-time monitoring tracks the agent's current state and alerts operators to anomalies:

- **Health checks**: Is the agent responsive? Is it making progress toward its goal?
- **Cost tracking**: Is the agent consuming tokens at an expected rate? Are there unexpected cost spikes?
- **Error rates**: What percentage of tool calls are failing? Are failure rates increasing?
- **Latency tracking**: How long is each step taking? Is latency increasing over time?

Real-time monitoring enables operators to detect and respond to problems before they impact the user. For example, if an agent enters an infinite loop (repeating the same action), the monitoring system can detect the pattern and trigger a restart.

Layer 2: Post-Hoc Analysis. Post-hoc analysis examines completed agent trajectories to identify patterns and improvement opportunities:

- **Success factor analysis**: What distinguishes successful trajectories from failed ones? Are there common decision points that correlate with success?
- **Cost optimization**: Where are the tokens going? Can any steps be eliminated, compressed, or parallelized?
- **Error pattern analysis**: What types of errors occur most frequently? Are there systematic weaknesses in the agent's reasoning or tool usage?

Post-hoc analysis drives continuous improvement of the harness, identifying opportunities to optimize prompts, add tools, improve state management, or strengthen guardrails.

Layer 3: Comparative Evaluation. Comparative evaluation assesses the impact of harness changes by comparing trajectories across versions:

- **A/B testing**: Run two versions of the harness on the same task set and compare performance.
- **Regression testing**: Verify that harness changes do not degrade performance on previously successful tasks.
- **Model comparison**: Compare the performance of different models within the same harness to inform model selection decisions.

This three-layer observability stack transforms agent development from an ad-hoc process into a data-driven engineering discipline, where every change is measured and validated against clear performance metrics.

5.4 Cost Optimization for Production Harnesses

Production deployment of agent systems introduces cost as a first-class engineering constraint. A single agentic task may consume 50,000–500,000 tokens across multiple LLM invocations, translating to significant API costs at scale.

5.4.1 Token Budget Management

The harness must implement token budget management to prevent cost overruns. Key strategies include:

1. **Task-level budgets**: Assign a maximum token budget to each task based on its complexity. If the agent approaches the budget limit, the harness triggers a graceful termination with a partial result.
2. **Step-level budgets**: Limit the token consumption of each step in the control loop. If a single LLM invocation exceeds the step budget, the harness truncates the output and retries with a more constrained prompt.
3. **Adaptive budgeting**: Dynamically adjust budgets based on the agent's progress. If the agent is making steady progress (as measured by verification milestones), the budget is increased. If the agent is stuck in a loop, the budget is decreased.

5.4.2 Prompt Caching and Context Optimization

Prompt caching—where the LLM provider caches repeated prompt prefixes and charges only for new tokens—can reduce costs by 80–90% for agentic workflows where the system prompt and tool schemas are constant across invocations. The harness should be designed to maximize cache hits by:

- Placing static content (system prompts, tool schemas, few-shot examples) at the beginning of the prompt.
- Appending new content (conversation history, tool outputs) at the end of the prompt.
- Avoiding modifications to the static prefix across invocations.

Context optimization reduces the number of tokens consumed per step. Strategies include compressing tool outputs (e.g., returning only the relevant lines from a file search rather than the entire file), summarizing conversation history (as discussed in Section 3.3), and using structured output formats (JSON, YAML) that are more token-efficient than natural language.

5.4.3 Model Routing and Cascade Strategies

Not every step in an agentic workflow requires the most capable (and most expensive) model. The harness can implement a *model routing* strategy that selects the appropriate model for each step:

1. **Fast model for routine steps**: Use a smaller, faster model (e.g., Claude Haiku, GPT-4o-mini) for simple tool calls, output parsing, and routine decisions.
2. **Capable model for complex steps**: Use the most capable model (e.g., Claude Opus, GPT-4) for planning, complex reasoning, and creative problem solving.
3. **Cascade verification**: Use a fast model for initial verification; escalate to a capable model only if the fast model flags a potential issue.

This cascade strategy can reduce costs by 40–60% while maintaining quality, as most agentic steps are routine operations that do not require frontier model capabilities.

5.5 Case Study: Production Harness Architectures

We examine three production harness architectures in detail to illustrate how the principles discussed in this section are applied in practice.

5.5.1 OpenHands: The Code-Centric Harness

OpenHands (Wang et al., 2024d) is an open-source platform for AI software engineering agents. Its harness architecture is notable for several design decisions:

1. **Code-based action space**: All agent actions are expressed as Python code, leveraging the CodeAct paradigm (Wang et al., 2024c). The agent writes code to search files, edit code, run tests, and communicate with the user.
2. **Sandboxed execution**: All code is executed in a Docker-based sandbox with network isolation, preventing the agent from making unintended changes to the host system.
3. **Event stream architecture**: All agent actions and observations are recorded as events in an immutable stream, enabling full replay and debugging.
4. **Multi-modal support**: The harness can process screenshots and visual output, enabling the agent to debug GUI applications.

The OpenHands harness demonstrates that the code-centric approach—where the agent's action space is a full programming language—is both more expressive and more reliable than the tool-calling approach for software engineering tasks.

5.5.2 SWE-agent: The ACI-Optimized Harness

SWE-agent (Yang et al., 2024) focuses on optimizing the Agent-Computer Interface (ACI) for software engineering. Key innovations include:

1. **Specialized search commands**: Custom file search and code search tools that return structured, relevant results rather than raw output.
2. **Context window management**: Automatic summarization of long file contents to fit within the context window while preserving relevant information.
3. **Demonstration-based prompting**: The system prompt includes demonstrations of successful problem-solving trajectories, teaching the agent effective strategies.

SWE-agent's success on SWE-bench demonstrates that careful ACI design—the tool interface component $\mathcal{T}$—can have a larger impact on performance than the underlying model.

5.5.3 Cursor: The IDE-Integrated Harness

Cursor represents a different approach: integrating the agent harness directly into the developer's IDE. This integration provides several advantages:

1. **Rich context**: The harness has direct access to the IDE's semantic information (syntax trees, type information, project structure), enabling more informed decisions.
2. **Human-in-the-loop**: The developer can review, modify, or reject agent actions in real-time, providing a natural guardrail system.
3. **Incremental workflow**: The agent suggests changes that are immediately visible in the editor, enabling rapid iteration and feedback.

The Cursor model—where the agent is embedded within the user's existing workflow rather than operating in a separate environment—represents an important direction for harness engineering. This approach reduces the friction of adoption and leverages the user's existing expertise as a safety mechanism.

6 The Solopreneur's Industrial Brain

The convergence of capable LLMs, standardized tool interfaces, and robust harness architectures is enabling a new paradigm: the *Industrial Brain* for individual knowledge workers. In this vision, every professional has a personal AI agent that manages their information, automates their workflows, and amplifies their cognitive capacity. This section explores how harness engineering principles apply to personal productivity.

6.1 PARA and GTD Frameworks for Agent Memory

The effectiveness of a personal AI agent depends critically on how it organizes and retrieves information. Two decades of productivity research provide proven frameworks that can be adapted for agent memory systems.

6.1.1 The PARA Method for Agent Knowledge Bases

The PARA method (Forte, 2022) organizes information into four categories:

1. **Projects**: Active endeavors with defined goals and deadlines (e.g., "Q1 Product Launch," "Tax Filing 2025").
2. **Areas**: Ongoing responsibilities without deadlines (e.g., "Health," "Finances," "Professional Development").
3. **Resources**: Topics of ongoing interest (e.g., "Machine Learning," "Design Patterns," "Travel").
4. **Archives**: Inactive items from the other three categories, preserved for future reference.

Adapting PARA for agent memory creates a structured knowledge base that maps naturally to the harness's state store $\mathcal{S}$. Each PARA category corresponds to a different memory management strategy:

- **Projects** receive high-priority, frequently updated storage with active retrieval. The agent maintains current state, pending tasks, and relevant context for each active project.
- **Areas** receive medium-priority storage with periodic updates. The agent tracks ongoing metrics, scheduled reviews, and relevant resources.
- **Resources** receive on-demand retrieval from a large knowledge store, using RAG techniques (Lewis et al., 2020) to retrieve relevant information only when needed.
- **Archives** receive compressed storage with efficient search, reducing storage costs while maintaining accessibility.

The PARA hierarchy also informs the context management strategy: project-related information is kept in working memory, area-related information is summarized, resource-related information is retrieved on demand, and archived information is accessed only when explicitly requested.

6.1.2 GTD Workflow Automation via Agents

David Allen's Getting Things Done (GTD) methodology (Allen, 2001) provides a five-phase workflow for managing personal productivity:

1. **Capture**: Collect all inputs (emails, messages, ideas, commitments) into a trusted system.
2. **Clarify**: Process each input to determine what it means and what action is required.
3. **Organize**: Assign clarified items to projects, contexts, or reference material.
4. **Reflect**: Periodically review the system to ensure completeness and relevance.
5. **Engage**: Take action on the most important items.

A harness-engineered agent can automate significant portions of this workflow. The capture phase can be fully automated: the agent monitors email, messaging, and document systems, extracting action items and commitments. The clarify phase can be partially automated: the LLM categorizes items, identifies action items, and drafts responses for human review. The organize phase maps directly to the PARA knowledge base structure. The reflect phase can be automated through scheduled reviews where the agent generates summaries and highlights items requiring attention. The engage phase remains primarily human-driven, with the agent providing context, drafting responses, and executing delegated tasks.

The key engineering challenge is maintaining alignment between the agent's understanding and the user's priorities. Unlike organizational AI systems where priorities are explicit (e.g., resolve GitHub issues, meet SLAs), personal productivity priorities are often implicit and evolving. The harness must incorporate preference learning mechanisms that adapt to the user's changing priorities over time.

6.1.3 Integration with Knowledge Management Systems

Modern knowledge management tools—Notion, Obsidian, Logseq—provide rich APIs that can serve as the persistence layer for agent memory systems. The integration architecture typically follows a hub-and-spoke model:

1. **Hub**: The agent harness manages the knowledge graph, handling storage, retrieval, and organization.
2. **Spokes**: External tools (Notion, Obsidian, email, calendar) connect through MCP (Anthropic, 2024b) servers that expose their data through the standardized interface.

This architecture enables the agent to maintain a unified view of the user's information across all tools, without requiring the user to change their existing workflows. The PARA structure is maintained within the hub, while the spokes provide access to tool-specific data (e.g., Notion databases, Obsidian markdown files, Google Calendar events).

6.2 The Digital Twin Paradigm

The concept of a digital twin—a computational replica that mirrors and predicts the behavior of a physical system—has been applied to manufacturing, urban planning, and healthcare. We extend this concept to personal AI agents: the *Personal Digital Twin* is an AI agent that maintains a comprehensive model of the user's knowledge, preferences, workflows, and goals.

6.2.1 Personal AI Agents as Digital Twins

The concept of using LLMs to construct digital twins that capture individual behavior patterns has gained increasing attention. In the harness context, the digital twin paradigm requires the state store $\mathcal{S}$ to maintain:

- **Preference model**: The user's communication style, decision patterns, and recurring choices.
- **Workflow templates**: The user's standard operating procedures for common tasks (e.g., "how I write blog posts," "how I review code").
- **Knowledge graph**: The user's domain expertise, including concepts, relationships, and mental models.
- **Goal hierarchy**: The user's short-term and long-term objectives, with progress tracking.

The Generative Agents framework (Park et al., 2023) provides a template for maintaining this comprehensive model. The agent continuously records observations about the user's behavior, reflects on patterns, and updates its internal model. Over time, the digital twin becomes increasingly accurate at predicting the user's needs and preferences.

6.2.2 Alignment and Preference Learning

The alignment challenge for personal digital twins is distinct from the general AI alignment problem. Rather than aligning with universal human values, the twin must align with a specific individual's preferences—which may be idiosyncratic, context-dependent, and evolving.

The harness implements preference learning through several mechanisms:

1. **Explicit feedback**: The user rates the agent's suggestions, providing direct preference signals.
2. **Implicit feedback**: The agent observes which suggestions the user accepts, modifies, or rejects.
3. **Behavioral modeling**: The agent analyzes the user's past decisions to infer preference patterns.

This preference learning loop is analogous to RLHF (Ouyang et al., 2022) but operates at the individual level rather than the population level. The harness must balance exploitation (using learned preferences to automate decisions) with exploration (suggesting new approaches to avoid stagnation).

6.2.3 Workflow Automation Case Studies

We examine three concrete applications of the digital twin paradigm:

Case Study 1: Automated Research Assistant. A researcher configures their digital twin with their research interests, reading preferences, and writing style. The agent:

1. Monitors arXiv and conference proceedings for relevant papers.
2. Reads and summarizes papers matching the researcher's interests.
3. Identifies connections between new papers and the researcher's existing work.
4. Drafts literature review sections in the researcher's writing style.

Case Study 2: Software Engineering Copilot. A developer's digital twin maintains a model of their codebase knowledge, preferred coding patterns, and review style. The agent:

1. Reviews pull requests with the developer's quality criteria.
2. Suggests code changes aligned with the project's architectural patterns.
3. Generates documentation in the developer's preferred format.
4. Monitors CI/CD pipelines and triages failures.

Case Study 3: Project Management Automation. A project manager's digital twin tracks project timelines, team capacity, and risk factors. The agent:

1. Monitors progress against milestones and flags delays.
2. Drafts status reports in the manager's communication style.
3. Suggests resource reallocation based on workload analysis.
4. Schedules and prepares agenda for recurring meetings.

In each case, the harness engineering challenges are similar: maintaining an accurate model of the user's preferences, integrating with diverse external tools through standardized interfaces, and providing appropriate autonomy with human oversight.

6.3 The Economics of the Industrial Brain

The viability of the Industrial Brain paradigm depends on economic factors as much as technical ones. We analyze the cost structure of personal AI agents and identify the conditions under which they become economically viable.

6.3.1 Cost Structure Analysis

The cost of operating a personal AI agent consists of several components:

1. **LLM inference costs**: The dominant cost, typically $0.50–5.00 per day for an active user, depending on the model and usage patterns. A software engineering agent that makes 50–100 LLM calls per day at $0.01–0.05 per call incurs $0.50–5.00 in LLM costs.
2. **Infrastructure costs**: Sandbox environments, databases, and MCP servers add $0.10–0.50 per day for cloud-hosted systems.
3. **Tool API costs**: External tool usage (e.g., web search, database queries) adds $0.05–0.50 per day depending on the tools used.
4. **Human oversight costs**: The time spent reviewing agent actions and providing feedback, estimated at 15–30 minutes per day for an active user.

The total cost of $1–6 per day translates to $30–180 per month. This is comparable to the cost of a human personal assistant for a few hours per month, but the AI agent is available 24/7 and can handle routine tasks in seconds rather than hours.

6.3.2 Value Creation and ROI

The value created by a personal AI agent depends on the type of work it automates:

- **Routine automation**: Email triage, meeting scheduling, document formatting. These tasks are low-value but time-consuming. An agent that saves 1–2 hours per day on routine tasks creates $50–100 per day in value (assuming $50/hour knowledge worker cost).
- **Knowledge amplification**: Literature review, code review, data analysis. These tasks are high-value and benefit from the agent's ability to process large volumes of information. An agent that improves knowledge worker productivity by 20–30% creates $100–150 per day in value.
- **Creative augmentation**: Drafting, brainstorming, prototyping. These tasks benefit from the agent's generative capabilities. The value is harder to quantify but can be substantial for creative professionals.

In all cases, the return on investment is strongly positive: $30–180 per month in costs versus $1,000–3,000 per month in value created. This economic analysis suggests that the primary barrier to adoption is not cost but trust and reliability—users must trust the agent to handle their personal information and make decisions on their behalf.

6.4 Toward the Autonomous Organization

The Industrial Brain paradigm scales beyond individual knowledge workers. When every member of an organization has a personal AI agent, the organization itself becomes a hybrid human-AI system. This "Autonomous Organization" vision has several implications for harness engineering.

6.4.1 Inter-Agent Communication

In an organization where every member has an AI agent, the agents must communicate with each other to coordinate work. This requires:

1. A standardized protocol for inter-agent communication (analogous to MCP for tool communication).
2. Permission models that respect organizational hierarchies and information security policies.
3. Conflict resolution mechanisms for when agents disagree on priorities or approaches.

The multi-agent coordination research discussed in Section 4.3 provides a foundation, but organizational deployment introduces additional constraints: agents must respect role boundaries, follow organizational policies, and maintain appropriate information boundaries.

6.4.2 Organizational Memory

An organization's collective knowledge—its processes, decisions, and lessons learned—is currently distributed across individual minds, documents, and systems. AI agents can create a unified organizational memory:

1. Each agent contributes observations and decisions to a shared knowledge base.
2. The knowledge base is organized using the PARA structure (Section 6.1) at the organizational level.
3. Agents query the organizational memory when making decisions, ensuring continuity and consistency.

This organizational memory addresses one of the most persistent problems in knowledge management: the loss of institutional knowledge when employees leave. With agent-mediated organizational memory, critical knowledge is captured and accessible regardless of personnel changes.

6.4.3 Governance and Accountability

The deployment of autonomous agents in an organizational context raises governance questions:

- Who is responsible when an agent makes a mistake?
- How should agents be audited for compliance with organizational policies?
- What limits should be placed on agent autonomy?
- How should agents handle confidential information?

These questions require both technical solutions (audit logging, permission systems, guardrails) and organizational solutions (policies, training, oversight structures). The harness engineering discipline must address both dimensions to enable responsible deployment of the Autonomous Organization vision.

7 Challenges and Future Directions

Despite the rapid progress documented in this survey, significant challenges remain before harness engineering can achieve the maturity of traditional software engineering disciplines. We identify three critical areas requiring further research and development.

7.1 The "Agentic OS" and Kernel-Level Harnesses

Current agent harnesses are built as application-layer software, analogous to user-space programs running on top of general-purpose operating systems. This architecture is suboptimal because the OS provides abstractions (processes, filesystems, networking) designed for human-managed workloads, not autonomous AI agents.

The concept of an "Agentic OS"—an operating system designed specifically for AI agent workloads—has emerged as a potential solution. An Agentic OS would provide:

1. **Agent-native process management**: First-class support for agent lifecycles, including checkpointing, migration, and inter-agent communication.
2. **Semantic filesystems**: Filesystems that organize information by meaning rather than path hierarchy, enabling agents to discover relevant resources through semantic queries.
3. **Capability-based security**: Security models based on agent capabilities rather than user identities, enabling fine-grained access control for autonomous systems.
4. **Resource-aware scheduling**: Schedulers that understand the latency and cost characteristics of LLM inference, optimizing for both performance and cost.

Recent developments point toward this vision. Anthropic's computer use capability (Anthropic, 2024a) enables agents to interact with desktop environments through mouse and keyboard actions, effectively treating the GUI as an API. OpenAI's Operator (OpenAI, 2025) provides a similar capability for web browsing. These systems suggest a trajectory toward agents that interact with computers the same way humans do—through graphical interfaces—which may prove more robust than API-based approaches for general-purpose automation.

MemGPT's (Packer et al., 2024) virtual memory system for LLMs is a step toward agent-native operating system features. By treating the context window as RAM and external storage as disk, MemGPT demonstrates that OS-level abstractions can be productively applied to agent memory management.

7.2 Standardization of Agent Evaluation (Evals)

The evaluation of agent systems remains fragmented, with no universally accepted benchmarks or methodologies. Current evaluation approaches include:

1. **Task-completion benchmarks**: SWE-bench (Jimenez et al., 2024) for software engineering, WebArena (Zhou et al., 2024) for web interaction, and AgentBench (Liu et al., 2024a) for general agent tasks. These measure end-to-end task success but provide limited insight into *why* agents succeed or fail.
2. **Capability benchmarks**: HumanEval (Austin et al., 2021) for code generation, MATH (Hendrycks et al., 2021) for mathematical reasoning. These evaluate specific skills but may not correlate with real-world agent performance.
3. **Framework-specific evaluations**: Each framework (OpenHands, SWE-agent, MetaGPT) defines its own evaluation suite, making cross-framework comparison difficult.

Arabzadeh et al. (2024) proposed AgentEval, a framework designed to simplify utility verification for LLM-powered applications by automatically proposing evaluation criteria tailored to each application's purpose. AgentEval quantifies application utility against the suggested criteria and provides tools for automated assessment.

The path toward standardized evaluation requires:

- **Unified task suites**: Benchmarks that span multiple domains and difficulty levels.

- **Fine-grained metrics**: Beyond binary success/failure, metrics that capture partial progress, efficiency, and resource consumption.
- **Reproducibility standards**: Standardized environments, seeds, and evaluation protocols.
- **Cost-normalized comparison**: Evaluation that accounts for the cost (tokens, API calls, latency) of achieving results.

7.3 Safety, Security, and Prompt Injection in the Harness

The harness layer is both the primary defense against and the primary target for adversarial attacks on agent systems. We analyze the security landscape along three dimensions.

7.3.1 Attack Surfaces in the Harness Layer

Agent harnesses introduce novel attack surfaces that do not exist in traditional LLM deployments:

1. **Indirect prompt injection**: Adversarial content embedded in tool outputs (e.g., web pages, documents, emails) that manipulates the agent's behavior (Greshake et al., 2023). Because the agent processes external data as part of its normal operation, indirect injection can bypass input sanitization.
2. **Tool poisoning**: Malicious MCP servers or tool implementations that return manipulated results to steer the agent toward harmful actions.
3. **Context window manipulation**: Techniques that fill the context window with irrelevant information, forcing the agent to "forget" critical instructions or constraints.
4. **Multi-agent manipulation**: In multi-agent systems, adversarial agents can inject misinformation that propagates through the agent network.

Liu et al. (2024b) demonstrated automatic and universal prompt injection attacks that achieve high success rates against major LLM-based agents. Their findings highlight the urgency of developing harness-level defenses.

7.3.2 Defense Strategies

Harness-level defense strategies include:

1. **Instruction hierarchy**: Explicitly ranking the authority of different context sources (system instructions $>$ user input $>$ tool output) and training the model to respect this hierarchy.
2. **Output validation**: The guardrail system $\mathcal{G}$ validates all agent outputs before execution, checking for policy violations, dangerous actions, and anomalous behavior patterns.
3. **Tool output sanitization**: Pre-processing tool outputs to remove or flag potentially adversarial content before injection into the LLM's context.
4. **Behavioral monitoring**: Continuous monitoring of agent behavior for signs of manipulation, such as sudden changes in action patterns, unusual tool usage, or deviation from the established plan.

Zou et al. (2023) demonstrated that universal adversarial attacks can be crafted to manipulate aligned language models, underscoring the need for multi-layered defenses. The constitutional AI approach (Bai et al., 2022)—where the model critiques its own outputs against a set of principles—provides a complementary defense at the model level, but is insufficient on its own for high-stakes deployments.

7.3.3 The Trust Boundary Problem

A fundamental challenge in harness security is defining the trust boundary between the agent and the external world. Every tool call crosses this boundary, and each crossing is a potential attack vector. The harness must balance two competing requirements:

1. **Utility**: The agent needs broad access to external tools to accomplish its tasks.

2. **Security**: Each additional tool and data source expands the attack surface.

This tradeoff is particularly acute for personal digital twins (Section 6.2), where the agent needs access to sensitive personal data (email, calendar, financial records) to provide useful automation. The harness must implement fine-grained permission management that grants access on a per-task basis while maintaining auditability and user control.

7.4 The Evolution of Tool Use: From APIs to GUIs

The trajectory of tool interfaces suggests a fundamental shift in how agents interact with computers. We identify three eras of tool use evolution:

7.4.1 Era 1: API-Based Tool Use (2023–2024)

The first generation of agent tools relied on structured APIs: function calling interfaces where the agent passes JSON parameters and receives structured responses. This era was defined by Toolformer (Schick et al., 2023), Gorilla (Patil et al., 2023), and ToolLLM (Qin et al., 2024). The API-based approach offers precision and reliability but is limited to services that expose programmatic interfaces.

The primary challenge of API-based tool use is coverage: the vast majority of software and websites do not expose APIs suitable for agent interaction. An agent that can only use APIs is fundamentally limited in its ability to interact with the digital world.

7.4.2 Era 2: GUI-Based Interaction (2024–2025)

The second generation of tool use enables agents to interact with graphical user interfaces through mouse clicks, keyboard input, and screenshot interpretation. Anthropic's computer use capability (Anthropic, 2024a) and OpenAI's Operator (OpenAI, 2025) represent this approach.

GUI-based interaction dramatically expands the agent's accessible tool space: any software with a graphical interface becomes a potential tool, regardless of whether it exposes an API. However, GUI interaction is inherently less reliable than API interaction because it requires visual understanding (interpreting screenshots), spatial reasoning (clicking the right location), and temporal reasoning (waiting for pages to load).

From a harness engineering perspective, GUI-based tool use requires:

1. **Visual grounding**: The harness must capture screenshots and make them available to the LLM for interpretation.
2. **Action execution**: The harness must translate the LLM's intended actions (e.g., "click the submit button") into concrete mouse and keyboard events.
3. **State tracking**: The harness must detect when the GUI state changes (e.g., a page loads, a dialog appears) and provide this information to the LLM.
4. **Error detection**: The harness must recognize when an action fails (e.g., an error dialog appears) and trigger recovery.

7.4.3 Era 3: Hybrid Tool Use (2025–present)

The emerging paradigm combines API-based and GUI-based tool use, selecting the most appropriate interface for each task. For services with well-designed APIs, the agent uses the structured interface for reliability. For services without APIs, the agent falls back to GUI interaction.

The hybrid approach introduces a new harness component: the *tool interface selector*, which determines the optimal interface for each tool invocation. This selector considers factors such as:

- Availability of an API for the desired operation.
- Reliability of the API versus the GUI approach.
- Latency and cost of each approach.
- Whether the operation requires visual judgment (favors GUI) or precise data manipulation (favors API).

The hybrid approach represents the most general and most challenging form of tool interface engineering, requiring the harness to manage multiple interaction paradigms simultaneously.

7.5 Open Problems in Harness Engineering

Beyond the specific challenges discussed above, several fundamental open problems remain in harness engineering.

7.5.1 The Specification Problem

How should users specify what they want an agent to do? Current approaches range from natural language instructions (which are ambiguous) to formal specifications (which are difficult to write). The harness must translate user intent into a form that the agent can reliably execute.

The specification problem is particularly acute for long-horizon tasks where the user cannot anticipate all the decisions the agent will need to make. The harness must balance between requesting clarification for every ambiguity (which is tedious) and making autonomous decisions (which may diverge from user intent).

7.5.2 The Composition Problem

How should harness components be composed? Current systems use ad-hoc integration: the orchestrator, tool interfaces, state management, and guardrails are tightly coupled. A more modular approach—where components can be swapped, upgraded, and reused independently—would accelerate development and enable systematic optimization.

The composition problem is related to the broader challenge of software architecture for AI systems. Traditional software architecture patterns (microservices, event-driven architecture, plugin systems) may provide useful templates, but the non-deterministic nature of LLM-based agents introduces unique challenges.

7.5.3 The Evaluation Gap

There is a significant gap between academic evaluation (benchmarks) and production evaluation (real-world performance). Agents that perform well on benchmarks may fail in production due to distribution shift, edge cases, and the complexity of real-world environments. Conversely, production-quality harnesses may prioritize robustness over benchmark performance, leading to lower scores on artificial tasks.

Closing this evaluation gap requires benchmarks that more closely approximate real-world conditions, including noisy inputs, incomplete information, changing requirements, and the need for human collaboration.

7.5.4 The Debugging Problem

Debugging agentic systems is fundamentally different from debugging traditional software. In traditional software, bugs are caused by incorrect logic; in agentic systems, "bugs" may be caused by the LLM's misunderstanding of the task, poor tool selection, context window limitations, or any number of other factors that are not code errors.

The debugging challenge requires new tools and techniques:

1. **Trajectory comparison**: Tools that compare successful and failed trajectories to identify divergent decisions.
2. **Ablation testing**: The ability to replace individual components (prompts, tools, models) to isolate the source of failures.
3. **Replay and modification**: The ability to replay a trajectory up to a specific point, modify the agent's state or the tool output, and observe the downstream effects.
4. **Attribution analysis**: Automated analysis that attributes failures to specific harness components (prompt, tool, state, control, guardrail).

7.5.5 The Scalability Problem

Current agent systems are designed for single-user, single-task execution. Scaling to multi-user, multi-task environments introduces challenges in resource management, state isolation, and cost allocation. The harness must be able to:

- Manage concurrent agent sessions without interference.
- Allocate computational resources (LLLM inference, tool execution) fairly across users.
- Track costs per user, per task, and per component for billing and optimization.
- Handle session persistence and recovery in the face of infrastructure failures.

These challenges point toward the need for agent-native infrastructure—the "Agentic OS" discussed in Section 7.1—that provides these capabilities as platform services rather than requiring each harness implementation to reinvent them.

8 Conclusion

Harness Engineering is the bridge between AI as a novelty and AI as a reliable industrial tool (Masterman et al., 2024; Wang et al., 2024b). This survey has demonstrated that the performance bottleneck in agentic AI systems has shifted from model capabilities to harness infrastructure, and that systematic engineering of the harness—the orchestration layer, tool interfaces, state management, and guardrails—is the primary determinant of real-world agent performance.

Our analysis reveals several key insights:

1. **Infrastructure over intelligence**: A well-engineered harness with a smaller model consistently outperforms a poorly engineered harness with a larger model. This finding, documented across software engineering (Jimenez et al., 2024; Yang et al., 2024), web interaction (Zhou et al., 2024), and general reasoning tasks, suggests that the returns to harness engineering investment are substantial and underappreciated.

2. **Standardization is accelerating**: The Model Context Protocol (Anthropic, 2024b) represents a critical inflection point in the maturation of the field. By decoupling tool implementation from agent frameworks, MCP enables the composability and reuse that characterize mature engineering disciplines.

3. **Verification is non-negotiable**: Production agent systems universally adopt some form of the Plan-Execute-Verify cycle (Wang et al., 2023a, 2024d). The verification phase—whether through automated testing, LLM-based critique, or human review—is what transforms a probabilistic model into a reliable agent.

4. **Memory architecture matters**: The distinction between working memory, episodic memory, and semantic memory—drawn from cognitive science and operationalized in systems like MemGPT (Packer et al., 2024) and Generative Agents (Park et al., 2023)—is critical for long-horizon tasks where context management determines success or failure.

5. **Security requires a harness-first approach**: As agents gain access to increasingly powerful tools and sensitive data, the harness layer becomes the primary security boundary. Defenses must be implemented at the harness level, complementing model-level alignment.

Looking forward, we anticipate the emergence of the "Agentic OS"—a purpose-built operating system for AI agent workloads—as the next major platform evolution. The standardization of evaluation benchmarks, the maturation of tool ecosystems, and the development of robust security frameworks will determine how quickly this vision becomes reality.

The field of Harness Engineering is young but evolving rapidly. We hope this survey provides a useful framework for researchers and practitioners working to build the next generation of reliable, secure, and effective AI agent systems.

Research Agenda

We conclude with a concrete research agenda for the next 2–3 years:

Short-term (2025–2026):

1. Develop standardized evaluation benchmarks that capture harness-level performance metrics (Section 4.5), not just end-task success.

2. Create open-source reference implementations of the five design patterns identified in Section 3.5, enabling systematic comparison.

3. Establish security evaluation frameworks for agent harnesses, analogous to OWASP for web applications.

4. Investigate the optimal division of labor between model-level and harness-level verification mechanisms.

Medium-term (2026–2027):

1. Build agent-native infrastructure (the "Agentic OS") that provides process management, memory management, and security as platform services.

2. Develop hybrid tool use systems (Section 7.4) that seamlessly combine API-based and GUI-based interaction.
3. Create personal digital twin systems (Section 6.2) with robust preference learning and alignment mechanisms.
4. Establish inter-agent communication standards for multi-agent organizational deployments (Section 6.4).

Long-term (2027+):

1. Develop the Autonomous Organization paradigm, where human-AI collaboration is the default mode of operation.
2. Create self-improving harnesses that optimize their own architecture based on accumulated experience.
3. Establish formal verification methods for agent systems that provide mathematical guarantees of safety and correctness.

A Detailed Timeline of Harness Engineering Milestones

Table 7 presents a chronological overview of the key milestones in the development of harness engineering for AI agents.

Table 7: Timeline of key milestones in harness engineering (2020–2026).

Year	Milestone	Significance
2020	GPT-3 (Brown et al., 2020)	Demonstrated few-shot learning; established the prompt as the primary user interface to LLMs.
2020	RAG (Lewis et al., 2020)	Introduced retrieval-augmented generation; established the paradigm for external knowledge integration.
2021	Scratchpad (Nye et al., 2021)	Showed that intermediate computation improves reasoning; precursor to chain-of-thought.
2021	HumanEval (Austin et al., 2021)	Established code generation as a key evaluation benchmark for LLMs.
2022	InstructGPT (Ouyang et al., 2022)	Demonstrated RLHF for instruction following; shifted focus from raw capability to controllability.
2022	Constitutional AI (Bai et al., 2022)	Introduced self-critique as a safety mechanism; foundation for model-level guardrails.
2022	WebGPT (Nakano et al., 2022)	First production system where an LLM browses the web; demonstrated the tool-use paradigm at scale.
2023	ReAct (Yao et al., 2023b)	Defined the Reason-Act-Observe loop; became the standard orchestration pattern for agents.
2023	Toolformer (Schick et al., 2023)	Showed that LLMs can learn to use tools through self-supervised learning.
2023	Reflexion (Shinn et al., 2023)	Introduced verbal reinforcement learning; demonstrated self-correcting agent loops.
2023	Generative Agents (Park et al., 2023)	Demonstrated memory, reflection, and planning for autonomous agents.
2024	SWE-bench (Jimenez et al., 2024)	Created the primary benchmark for software engineering agents.
2024	Voyager (Wang et al., 2024a)	Demonstrated skill library composition for open-ended learning.
2024	MCP (Anthropic, 2024b)	Introduced the first open standard for agent-tool communication.
2024	OpenHands (Wang et al., 2024d)	Established code-centric agent architecture for software engineering.
2024	O1 (OpenAI, 2024)	Demonstrated test-time scaling; shifted focus from training compute to inference compute.
2025	Computer Use (Anthropic, 2024a)	Enabled agents to interact with GUIs; expanded the tool interface beyond APIs.

This timeline reveals several patterns. First, the field has accelerated dramatically: 14 of the 17 milestones occurred in 2023–2025. Second, the focus has shifted from model capabilities (2020–2022) to harness infrastructure (2023–2025). Third, standardization efforts (MCP) are a recent phenomenon, suggesting that the field is entering a maturation phase.

B Comprehensive Comparison of Agent Frameworks

Table 8 provides a comprehensive comparison of the major agent frameworks discussed in this survey, evaluated across multiple dimensions.

Table 8: Comprehensive comparison of agent frameworks.

Framework	Year	Orch.	Tools	Memory	Verify	Open
ReAct (Yao et al., 2023b)	2023	Linear	Function	History	None	Yes
Reflexion (Shinn et al., 2023)	2023	Linear	Function	Reflect	Self	Yes
Voyager (Wang et al., 2024a)	2024	Linear	Code	Skills	Exec	Yes
MetaGPT (Hong et al., 2024)	2024	Hierarchical	Function	Shared	Review	Yes
ChatDev (Qian et al., 2024)	2024	Ring	Function	Shared	Test	Yes
AutoGen (Wu et al., 2023)	2023	Mesh	Custom	Shared	Custom	Yes
OpenHands (Wang et al., 2024d)	2024	Linear	Code	Events	Exec	Yes
SWE-agent (Yang et al., 2024)	2024	Linear	ACI	History	Test	Yes
HuggingGPT (Shen et al., 2023)	2024	Star	Models	None	Model	Yes
CrewAI (CrewAI, 2024)	2024	Star	Function	Shared	Custom	Yes
MemGPT (Packer et al., 2024)	2024	Linear	Function	Virtual	None	Yes

Key observations from this comparison:

1. **Linear orchestration dominates**: Despite the theoretical advantages of DAG-based and hierarchical orchestration, most production systems use the linear ReAct loop. This suggests that the simplicity and debuggability of linear loops outweigh their theoretical limitations in practice.

2. **Code-based tools are emerging**: The most successful recent systems (OpenHands, Voyager) use code as the action space rather than predefined function calls. This trend aligns with the CodeAct paradigm (Wang et al., 2024c).

3. **Verification is domain-dependent**: Software engineering systems use execution-based verification (running tests), while general-purpose systems rely on LLM-based self-critique. The choice of verification mechanism is a critical design decision for any harness.

4. **Memory is underdeveloped**: Most frameworks use simple conversation history for state management. Only MemGPT and Voyager implement sophisticated memory architectures, suggesting significant room for improvement.

C Harness Component Design Checklist

Based on our analysis, we provide a practical checklist for engineers designing agent harnesses.

C.1 Prompt System ($\mathcal{P}$) Design

1. Define a clear instruction hierarchy: system instructions > user input > tool output.
2. Include few-shot demonstrations of successful agent behavior.
3. Specify the agent's role, capabilities, and limitations explicitly.
4. Implement dynamic prompt construction based on the current task state.
5. Version-control all prompt templates alongside the codebase.

C.2 Tool Interface ($\mathcal{T}$) Design

1. Prefer MCP (Anthropic, 2024b) for tool integration to ensure interoperability.
2. Design tool schemas with clear parameter descriptions and examples.
3. Implement graceful degradation for tool failures (retry, fallback, user notification).
4. Validate tool outputs before injection into the LLM's context.
5. Consider the code-as-action paradigm (Wang et al., 2024c) for complex tool composition.

C.3 State Store ($\mathcal{S}$) Design

1. Implement a three-tier memory architecture: working, compressed, and long-term.
2. Use RAG (Lewis et al., 2020) for large knowledge stores; use structured storage for active state.
3. Implement context folding with information-priority-aware summarization.
4. Persist state across sessions for long-running tasks.
5. Log all state transitions for debugging and replay.

C.4 Control Loop ($\mathcal{C}$) Design

1. Start with a linear ReAct loop; upgrade to DAG-based orchestration only when needed.
2. Implement the PEV cycle (Section 5.1) for production systems.
3. Set maximum iteration limits to prevent infinite loops.
4. Include checkpoints where the agent reassesses its plan and progress.
5. Implement adaptive compute allocation based on task difficulty.

C.5 Guardrail ($\mathcal{G}$) Design

1. Validate all agent outputs before execution.
2. Implement rate limiting and cost budgets.
3. Include human-in-the-loop checkpoints for high-stakes decisions.
4. Monitor for signs of prompt injection (Greshake et al., 2023) and adversarial manipulation.
5. Maintain audit logs of all agent actions for forensics and compliance.

D Extended Analysis of Security Threats

This appendix provides an extended analysis of the security threats facing agent harnesses, organized by the attack surface.

D.1 Threat Model

We consider an adversary who can:

1. Inject content into tool outputs (e.g., through web pages, documents, or email bodies).
2. Provide malicious MCP servers or tool implementations.
3. Observe the agent's public actions (e.g., API calls, web requests).
4. Manipulate the agent's context through social engineering of the user.

The adversary's goals may include: extracting sensitive information from the agent's state store, causing the agent to perform unauthorized actions, degrading the agent's performance (denial of service), or propagating malicious instructions to other agents in a multi-agent system.

D.2 Attack Taxonomy

Table 9 categorizes the primary attack vectors against agent harnesses.

Table 9: Taxonomy of attacks against agent harnesses.

Attack Vector	**Target**	**Description**
Indirect Injection	$\mathcal{P}$, $\mathcal{S}$	Malicious instructions embedded in tool outputs that override the agent's system prompt.
Tool Poisoning	$\mathcal{T}$	Malicious tool implementations that return manipulated results or execute unauthorized actions.
Context Flooding	$\mathcal{S}$	Overwhelming the context window with irrelevant information to push out critical instructions.
Plan Hijacking	$\mathcal{C}$	Manipulating the agent's plan through carefully crafted tool outputs that suggest alternative actions.
Guardrail Bypass	$\mathcal{G}$	Crafting outputs that technically comply with guardrail rules while achieving the adversary's goal.
Model Extraction	$\mathcal{H}$	Using the agent's responses to reconstruct the system prompt or extract training data (Carlini et al., 2021).

D.3 Defense in Depth

We recommend a defense-in-depth strategy that implements protections at every layer of the harness:

1. **Model level**: Use models trained with constitutional AI (Bai et al., 2022) that resist adversarial manipulation.
2. **Prompt level**: Implement clear instruction hierarchies that prevent tool outputs from overriding system instructions.
3. **Tool level**: Sanitize all tool outputs before injection; use MCP's process isolation to limit the blast radius of compromised tools.
4. **State level**: Encrypt sensitive state; implement access controls for the state store.

5. **Control level**: Monitor for anomalous action patterns; implement circuit breakers that halt the agent if suspicious behavior is detected.
6. **Guardrail level**: Validate all outputs against a comprehensive policy; implement human-in-the-loop checkpoints for high-risk actions.

The key principle is that no single defense is sufficient; the harness must implement overlapping protections that together provide robust security even when individual defenses fail.

E The Future of Human-Agent Collaboration

The ultimate goal of harness engineering is not to replace humans but to create effective human-agent partnerships. This appendix explores the emerging paradigms for human-agent collaboration and their implications for harness design.

E.1 Collaboration Modes

We identify five distinct modes of human-agent collaboration, each with different requirements for the harness:

E.1.1 Mode 1: Agent as Tool

In this mode, the agent operates as a sophisticated tool that the human invokes for specific tasks. The human maintains full control: they decide when to invoke the agent, what task to assign, and whether to accept the result. The harness must provide:

- A clear interface for task specification and result delivery.
- Fast execution with minimal latency between invocation and result.
- Transparent operation so the human can understand and verify the result.

This mode is exemplified by AI-powered code completion tools (GitHub Copilot, Cursor's tab completion) where the agent generates suggestions that the human can accept, modify, or reject.

E.1.2 Mode 2: Agent as Assistant

In this mode, the agent proactively assists the human by anticipating needs, suggesting actions, and handling routine tasks autonomously. The human delegates tasks but retains oversight:

- The agent proposes actions rather than executing them immediately.
- The human reviews and approves proposed actions before execution.
- The agent learns from the human's approvals and rejections to improve future suggestions.

This mode is exemplified by email triage agents that categorize incoming messages, draft responses, and present them for human review. The harness must implement a robust approval workflow and maintain a model of the human's preferences.

E.1.3 Mode 3: Agent as Collaborator

In this mode, the agent and the human work together as peers, each contributing their strengths to a shared task. The agent might handle information retrieval, data analysis, and draft generation, while the human provides judgment, creativity, and domain expertise:

- The collaboration is *symmetric*: both the agent and the human can propose actions, ask questions, and make decisions.
- The harness must manage a shared workspace where both the agent and the human can contribute.
- Conflict resolution mechanisms are needed when the agent and the human disagree.

This mode is exemplified by collaborative writing tools where the agent drafts sections, the human edits them, and the agent incorporates the edits into subsequent drafts.

E.1.4 Mode 4: Agent as Delegate

In this mode, the human delegates entire tasks to the agent and reviews only the final result:

- The agent operates with significant autonomy, making decisions without human oversight.
- The harness must provide robust guardrails to ensure the agent stays within acceptable boundaries.
- The verification phase of the PEV cycle becomes critical as the primary quality gate.

This mode is exemplified by autonomous software engineering agents that are assigned bug fixes and submit pull requests for human review.

E.1.5 Mode 5: Agent as Supervisor

In this mode, the agent monitors the human's work, provides guidance, and catches errors:

- The agent observes the human's actions and identifies potential mistakes or improvements.
- The harness must implement observation capabilities (e.g., monitoring keystrokes, file changes, or command execution).
- The agent must be careful not to be intrusive; suggestions should be timely and relevant.

This mode is exemplified by AI-powered code review tools that analyze code as it is written and suggest improvements in real-time.

E.2 Trust Calibration

A critical challenge in human-agent collaboration is *trust calibration*: ensuring that the human's trust in the agent is appropriately matched to the agent's actual capability. Under-trust leads to underutilization (the human does not delegate enough tasks to the agent); over-trust leads to errors (the human accepts incorrect agent outputs).

The harness can facilitate trust calibration through several mechanisms:

1. **Confidence reporting**: The agent reports its confidence level for each action, enabling the human to calibrate their trust based on the agent's self-assessment.
2. **Transparency**: The agent explains its reasoning for each action, enabling the human to verify the logic rather than trusting the output blindly.
3. **Progressive autonomy**: The harness gradually increases the agent's autonomy as the human builds trust through successful interactions.
4. **Error disclosure**: When the agent makes a mistake, the harness proactively discloses the error and explains what went wrong, building honest trust rather than false confidence.

E.3 Design Principles for Collaborative Harnesses

Based on our analysis of collaboration modes and trust calibration, we propose five design principles for harnesses that support effective human-agent collaboration:

1. **Transparency by default**: All agent reasoning and actions should be visible to the human, unless the human explicitly opts for reduced transparency.
2. **Progressive control**: The harness should support a spectrum from full human control to full agent autonomy, and allow the human to adjust the level of control dynamically.
3. **Error tolerance**: The harness should be designed to handle human errors (e.g., incorrect approvals) and agent errors (e.g., incorrect suggestions) gracefully, with undo capabilities and recovery mechanisms.
4. **Learning from interaction**: The harness should learn from human-agent interactions to improve future collaboration, adapting to the human's preferences, working style, and trust level.
5. **Contextual awareness**: The harness should adapt its collaboration mode based on the context—offering more autonomy for routine tasks and more transparency for novel or high-stakes tasks.

F Formal Methods for Agent Verification

As agent systems are deployed in high-stakes environments (healthcare, finance, autonomous vehicles), informal verification through testing and LLM-based critique becomes insufficient. This appendix explores the application of formal methods to agent verification.

F.1 Specification Languages for Agent Behavior

Formal verification begins with a formal specification of desired behavior. For agent systems, we need to specify:

1. **Safety properties**: What the agent must never do (e.g., "never delete a file without user confirmation"). These are typically expressed as linear temporal logic (LTL) formulas.
2. **Liveness properties**: What the agent must eventually do (e.g., "the agent must complete the assigned task within the budget"). These are also expressed as LTL formulas.
3. **Invariants**: Properties that must hold at every step (e.g., "the agent's context window must never exceed the model's limit"). These can be checked at runtime.

The challenge is that agent behavior is inherently non-deterministic: the same prompt may produce different outputs on different invocations. Formal methods must account for this non-determinism by verifying properties over all possible behaviors, not just observed behaviors.

F.2 Runtime Verification

Runtime verification monitors the agent's execution against formal specifications, flagging violations in real-time. This approach does not guarantee correctness (it can only detect violations that occur during execution) but provides a practical balance between rigor and usability.

For agent harnesses, runtime verification can be implemented as part of the guardrail system $\mathcal{G}$:

1. **Pre-action verification**: Before executing each action, check it against safety specifications. If the action would violate a safety property, block it and report the violation.
2. **Post-action verification**: After each action, check whether the resulting state satisfies the invariants. If a violation is detected, initiate recovery.
3. **Trajectory verification**: After the agent completes a task, verify that the entire execution trace satisfies the liveness properties. This provides end-to-end assurance.

F.3 Type Systems for Agent Actions

An emerging approach applies type system concepts to agent actions. In this framework, each agent action has a "type" that describes its effects:

- **Pure actions**: Actions that do not modify external state (e.g., reading a file). These are safe to execute without verification.
- **Impure actions**: Actions that modify external state (e.g., writing a file, sending an email). These require verification before execution.
- **Destructive actions**: Actions that cannot be easily undone (e.g., deleting a file, deploying to production). These require human approval.

The harness can use this type system to automatically determine the appropriate verification level for each action. This approach is analogous to the effect systems used in functional programming languages to track side effects.

F.4 Contract-Based Agent Design

Contract-based design, borrowed from software engineering, specifies preconditions, postconditions, and invariants for each agent action:

- **Precondition**: What must be true before the action is executed (e.g., "the file must exist before it can be edited").
- **Postcondition**: What must be true after the action is executed (e.g., "the file must contain the specified changes").
- **Invariant**: What must be true before and after the action (e.g., "the project must still compile").

The harness checks these contracts at runtime, providing automatic verification that the agent's actions have the intended effects. Contract violations trigger error recovery or human escalation.

This approach is particularly powerful when combined with the MCP tool interface: MCP servers can expose contracts for their tools, enabling the harness to automatically verify that the agent's tool usage is correct.

F.5 Testing Methodologies for Agent Harnesses

Traditional software testing methodologies must be adapted for the non-deterministic nature of agent systems. We propose a testing pyramid for agent harnesses:

Unit Tests: Component-Level Testing. Each harness component ($\mathcal{P}$, $\mathcal{T}$, $\mathcal{S}$, $\mathcal{C}$, $\mathcal{G}$) is tested in isolation:

- **Prompt system tests**: Verify that prompts are correctly constructed for various task states. Use deterministic models or mock LLM responses.
- **Tool interface tests**: Verify that tool calls are correctly formatted and that responses are correctly parsed. Use mock tool servers.
- **State store tests**: Verify that state is correctly saved, retrieved, and compressed. Test edge cases like context overflow and state corruption.
- **Control loop tests**: Verify that the orchestration logic correctly handles various action types, errors, and completion conditions.
- **Guardrail tests**: Verify that safety checks correctly block dangerous actions and allow safe ones. Test adversarial inputs.

Integration Tests: System-Level Testing. The complete harness is tested with a real LLM on a curated set of tasks:

- **Happy path tests**: Standard tasks that the agent should complete reliably.
- **Error recovery tests**: Tasks where tool calls fail or produce unexpected results.
- **Edge case tests**: Tasks that stress specific harness components (e.g., tasks requiring many tool calls, tasks with large context).

Regression Tests: Version Comparison. Automated test suites that run on every harness change:

- **Behavioral regression**: The agent should produce similar (not identical) trajectories on the same tasks after a harness update.
- **Performance regression**: Task completion rates, token consumption, and latency should not degrade.
- **Safety regression**: The agent should not exhibit new unsafe behaviors.

Chaos Tests: Resilience Testing. Deliberately introduce failures to test the harness's resilience:

- **Network failures**: Simulate tool server outages and verify graceful degradation.
- **LLLM failures**: Simulate LLM API errors (rate limiting, timeouts, malformed responses).
- **State corruption**: Simulate state store failures and verify recovery.

This testing pyramid ensures that agent harnesses are reliable at every level, from individual components to the complete system operating under adverse conditions.

References

David Allen. *Getting Things Done: The Art of Stress-Free Productivity*. Penguin Books, 2001.

Anthropic. Computer use with claude, 2024a. URL https://docs.anthropic.com/en/docs/build-with-claude/computer-use.

Anthropic. Model context protocol, 2024b. URL https://modelcontextprotocol.io/.

Anthropic. Tool use with claude, 2024c. URL https://docs.anthropic.com/en/docs/build-with-claude/tool-use.

Negar Arabzadeh, Julia Kiseleva, Qingyun Wu, Chi Wang, Ahmed Awadallah, Victor Dibia, Adam Fourney, and Charles Clarke. Towards better human-agent alignment: Assessing task utility in LLM-powered applications. *arXiv preprint arXiv:2402.09015*, 2024.

Akari Asai, Zeqiu Wu, Yiran Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *International Conference on Learning Representations (ICLR)*, 2024.

Jacob Austin, Daniel D. Johnson, Maarten Jones, Alexander Toshev, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Priest, Tatjana Barakat, Aidan Buhmann, Sergei Bulatov, et al. Improving language models by retrieving from trillions of tokens. *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1877–1901, 2020.

Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, and Colin Raffel. Extracting training data from large language models. *Proceedings of the 30th USENIX Security Symposium*, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Cognition Labs. Devin: The first ai software engineer, 2024. URL https://www.cognition.ai/blog/devin-generally-capable-ai-software-engineer.

CrewAI. Crewai: Framework for orchestrating role-playing autonomous ai agents, 2024. URL https://github.com/crewAIInc/crewAI.

E2B. E2B: Secure sandboxed environments for ai agents, 2024. URL https://e2b.dev/.

Tiago Forte. *Building a Second Brain: A Proven Method to Organize Your Digital Life and Unlock Your Creative Potential*. Atria Books, 2022.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christof Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *Proceedings of the ACM Conference on Computer and Communications Security (ACM CCS)*, 2023.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *Proceedings of the Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. *International Conference on Learning Representations (ICLR)*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Xiang Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

Sehoon Kim et al. An llm compiler for parallel function calling. *International Conference on Machine Learning (ICML)*, 2024.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33: 9459–9474, 2020.

Xiao Liu et al. AgentBench: Evaluating llms as agents. *International Conference on Learning Representations (ICLR)*, 2024a.

Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. Automatic and universal prompt injection attacks against large language models. *arXiv preprint arXiv:2403.04957*, 2024b.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.

Tula Masterman, Sérgio Besen, Mason Sawtell, and Alex Chao. The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584*, 2024.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2022.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Jones, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.

OpenAI. Learning to reason with llms. *OpenAI Blog*, 2024. URL https://openai.com/index/learning-to-reason-with-llms/.

OpenAI. Introducing operator, 2025. URL https://openai.com/index/introducing-operator/.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Christopher Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35: 27730–27744, 2022.

Charles Packer, Vivian Fang, Anton Chaffin, Alon Argueta, Paul Cerrato, Joseph E. Gonzalez, Scott Katz, and Karthik Narasimhan. Memgpt: Towards llms as operating systems. *International Conference on Learning Representations (ICLR)*, 2024.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Shirriff, et al. Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.

Chen Qian et al. Chatdev: Communicative agents for software development. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lu Yan, Yujia Lu, Yankai Lin, Xingpei Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *International Conference on Learning Representations (ICLR)*, 2024.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.

Yongliang Shen, Kaitao Song, et al. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research (TMLR)*, 2024a.

Lei Wang, Wanyu Xu, et al. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2609–2634, 2023a.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186232, 2024b.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. *International Conference on Machine Learning (ICML)*, 2024c.

Xingyao Wang et al. OpenHands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024d.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations (ICLR)*, 2023b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:24824–24837, 2022.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yi Ding, Boyang Hong, Mingyu Zhang, Junzhe Wang, Wenyu Wang, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2024.

Guangxuan Xiao, Wenhan Yu, Tianyu An, Song Song, and Song Han. Efficient streaming language models with attention sinks. *International Conference on Learning Representations (ICLR)*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Shunyu Yao. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023b.

Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhu Chen, and Xiang Yue. OpenCodeInterpreter: Integrating code generation with execution and refinement. *Findings of the Association for Computational Linguistics (ACL Findings)*, 2024.

Denny Zhou, Nathanael Scharli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. *International Conference on Learning Representations (ICLR)*, 2023.

Shuyan Zhou et al. WebArena: A realistic web environment for building autonomous agents. *International Conference on Learning Representations (ICLR)*, 2024.

Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.